



CUDA Case Study

谢旻晖

信息楼505



中國人民大學
RENMIN UNIVERSITY OF CHINA

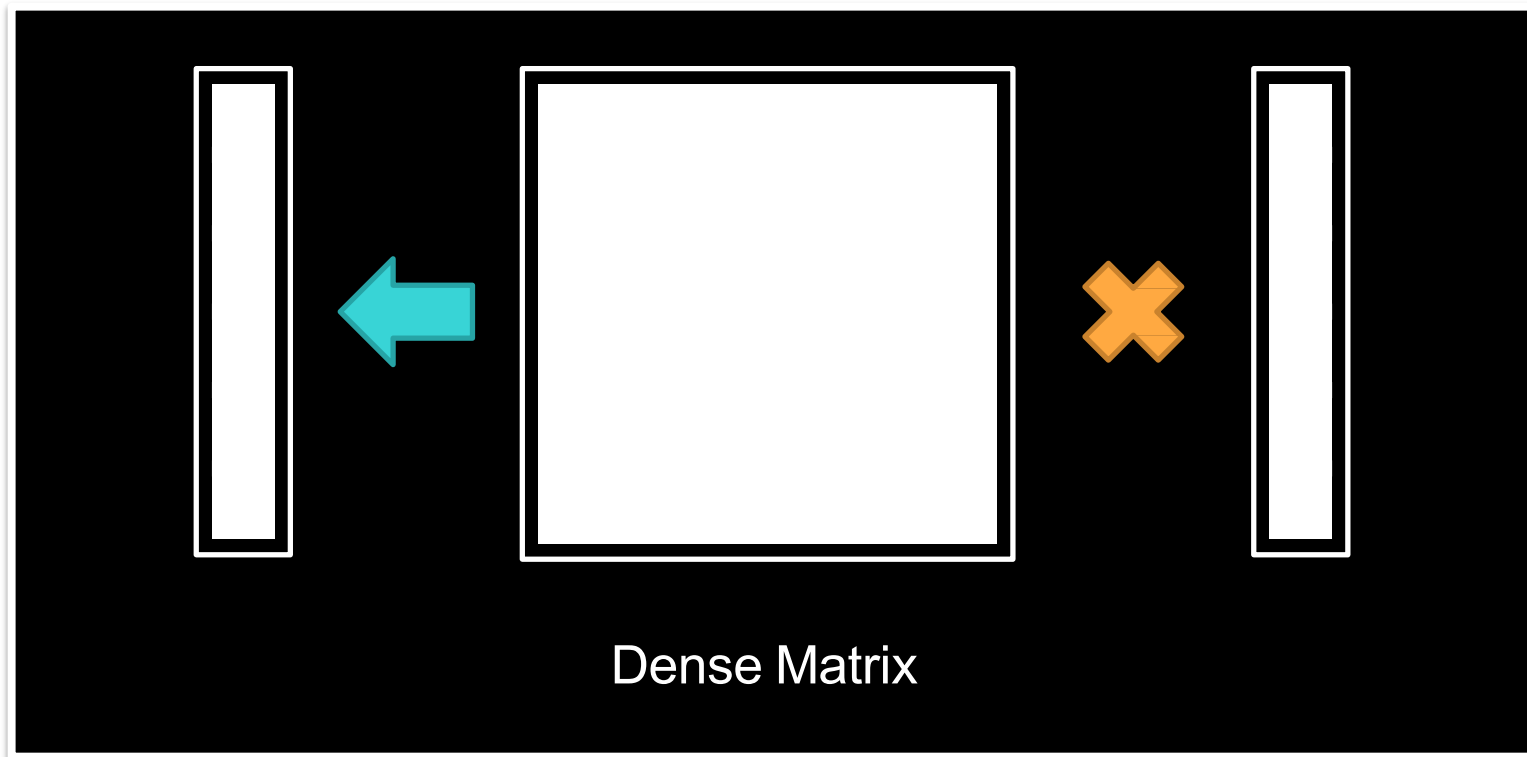


提纲

- 稀疏矩阵向量乘SpMV
- 性能考虑因素
- SpTRSV

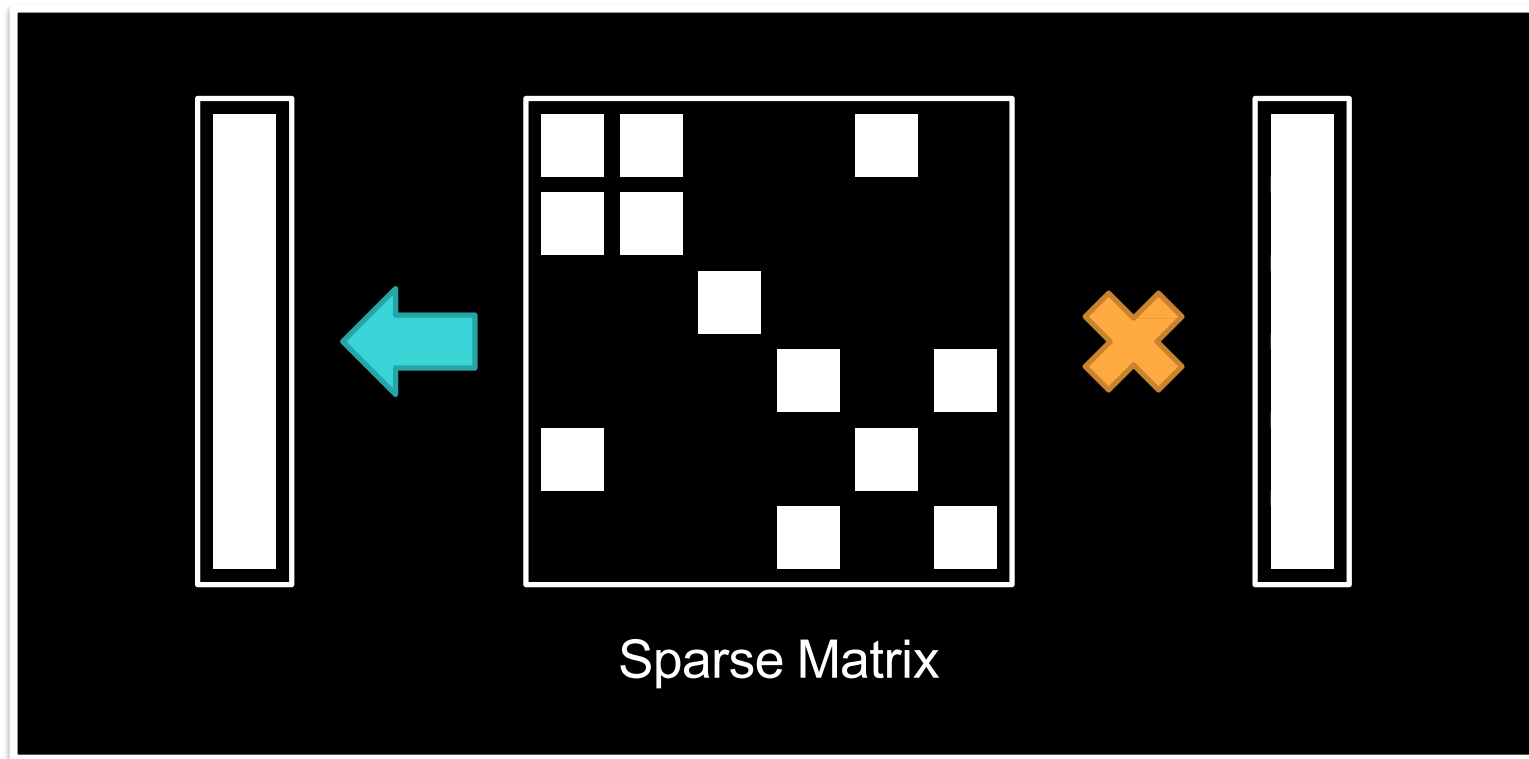
稠密矩阵向量乘法

- SGEMV/DGEMV in BLAS
 - memory-bound performance



稀疏矩阵向量乘法

- 每个非零元需要做 乘 / 加 运算
- **vector**数据重用





性能

- 性能瓶颈在于访存 (memory-bound)
 - 通常5-20 GFLOPS/s
- 计算密度低
 - 2 flops: 8+ bytes (float)
 - 2 flops: 16+ bytes (double)
- 最主要手段
 - 合理使用内存带宽



性能

- Tesla A100 浮点性能 (floating point)
 - Single 19.5 TFLOP/s (peak)
 - Double 9.7 TFLOP/s (peak)
- Tesla A100 HBM内存带宽
- 1935 GB/s (peak)
- 计算密度阈值 (intensity threshold)
 - 10.08 FLOP : Byte (single)
 - 5.01 FLOP : Byte (double)



性能

Tesla A100 阈值

10.08 FLOP : Byte (single)

5.01 FLOP : Byte (double)

稠密矩阵向量乘法计算密度

0.25 - 0.50 FLOP : Byte (single)

0.12 - 0.25 FLOP : Byte (double)

稀疏矩阵向量乘法计算密度

0.12 - 0.50 FLOP : Byte (single)

0.08 - 0.25 FLOP : Byte (double)

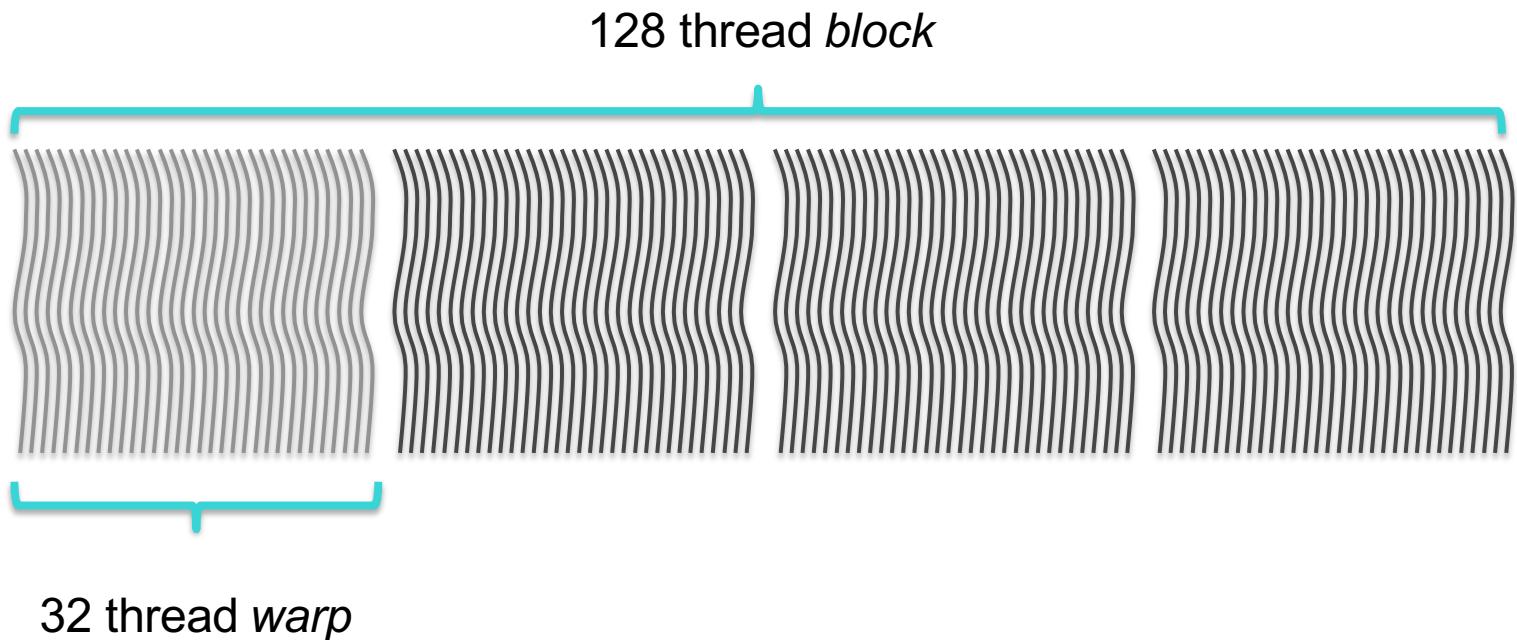


性能考虑因素

- 高效利用内存带宽
 - memory coalescing
- 细粒度并行
 - 数以千计的线程
- 尽可能多的数据重用
 - 利用缓存、shared memory

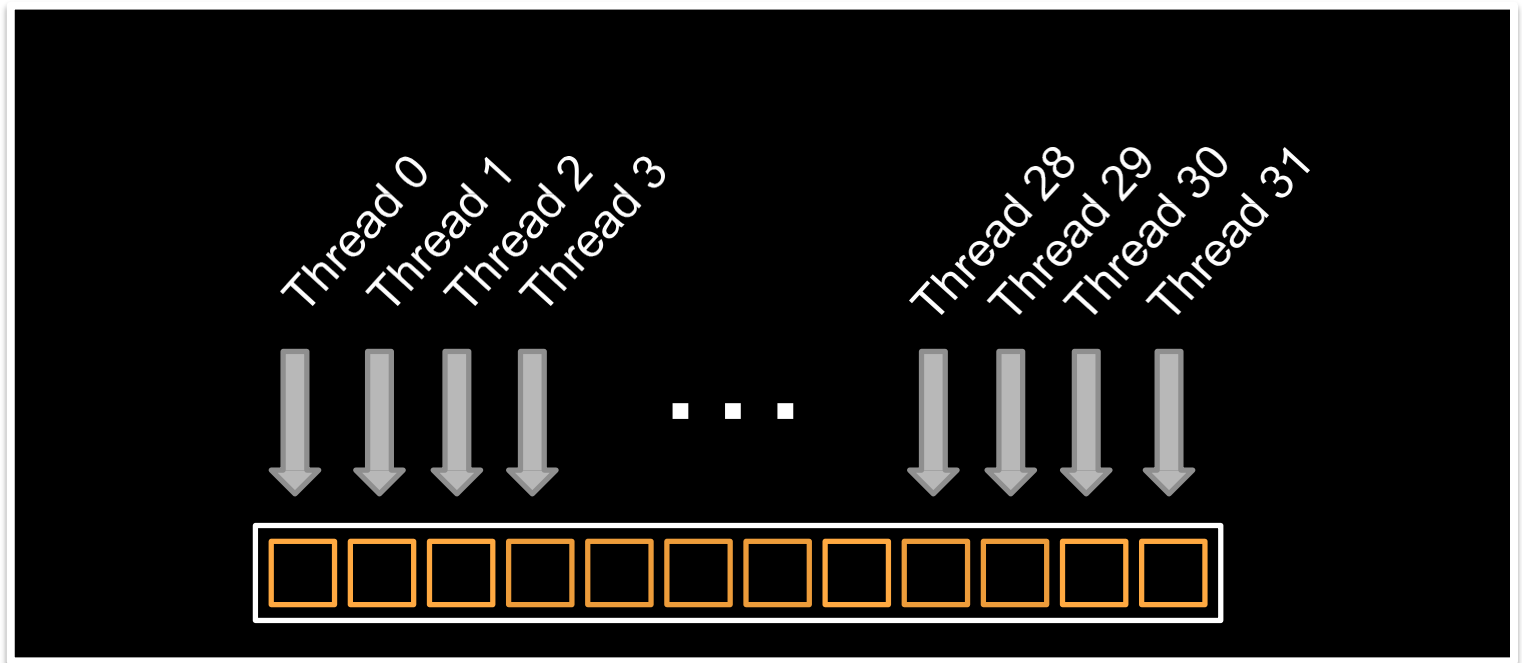
memory coalescing

- block被划分为物理warp来执行



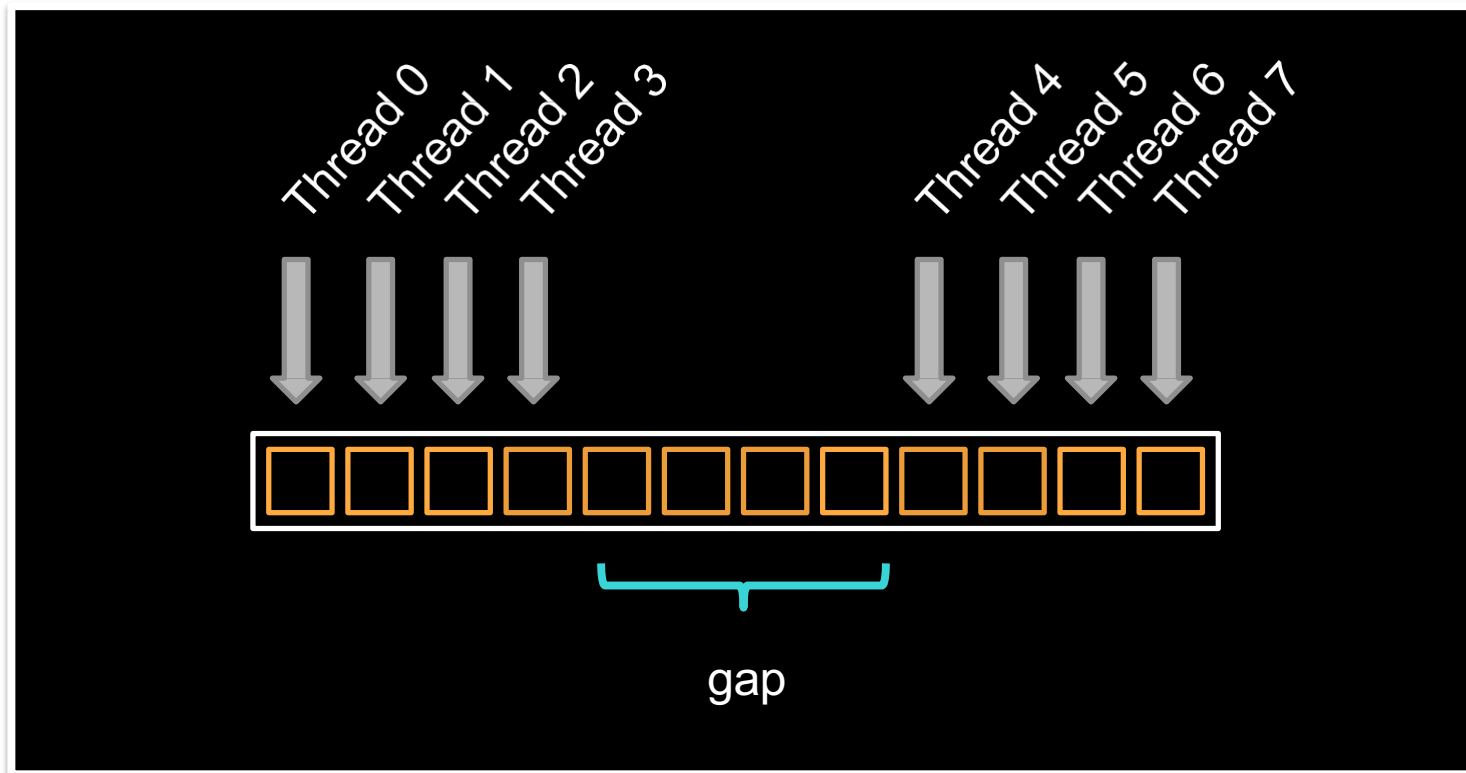
memory coalescing

- **fully** coalesced memory access



memory coalescing

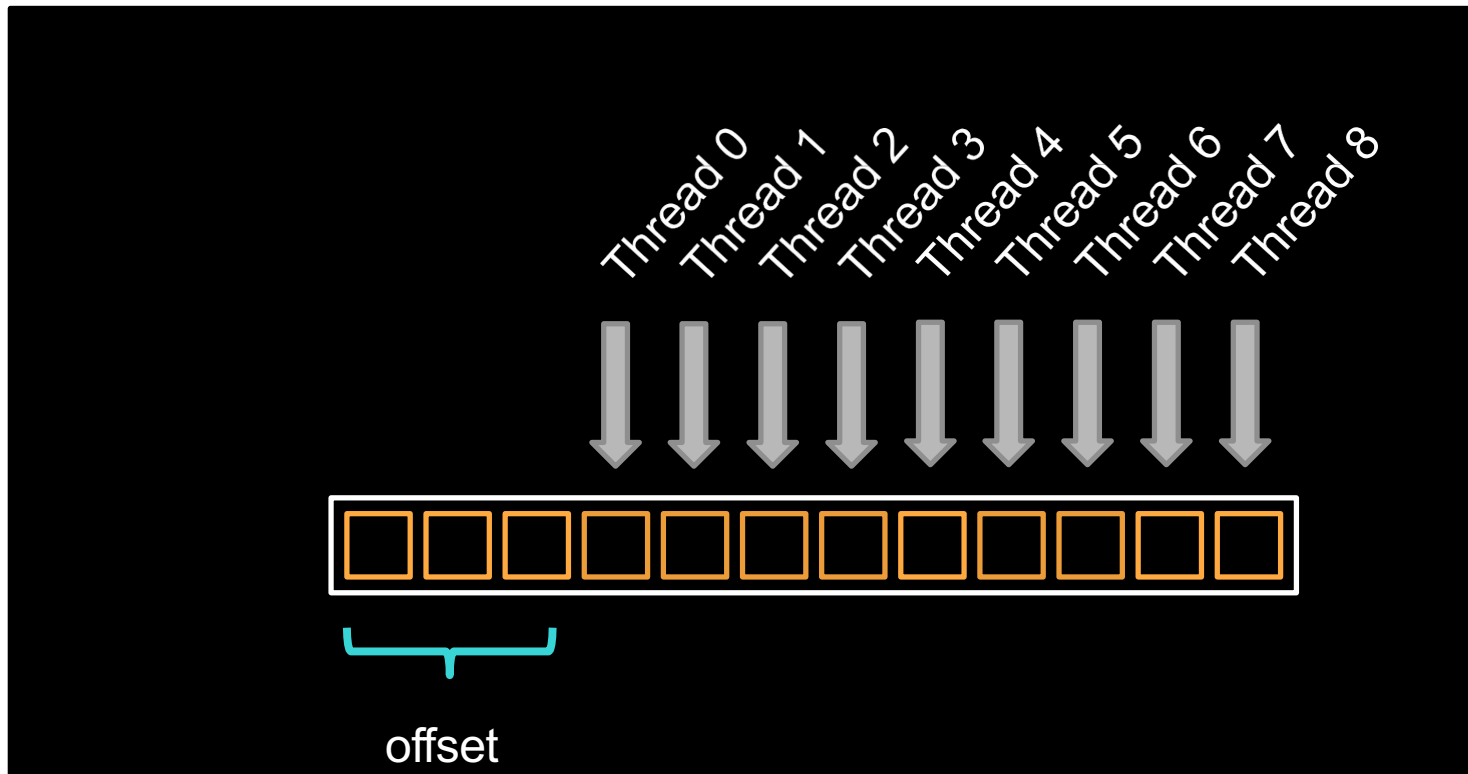
- **partially** coalesced memory access



memory coalescing

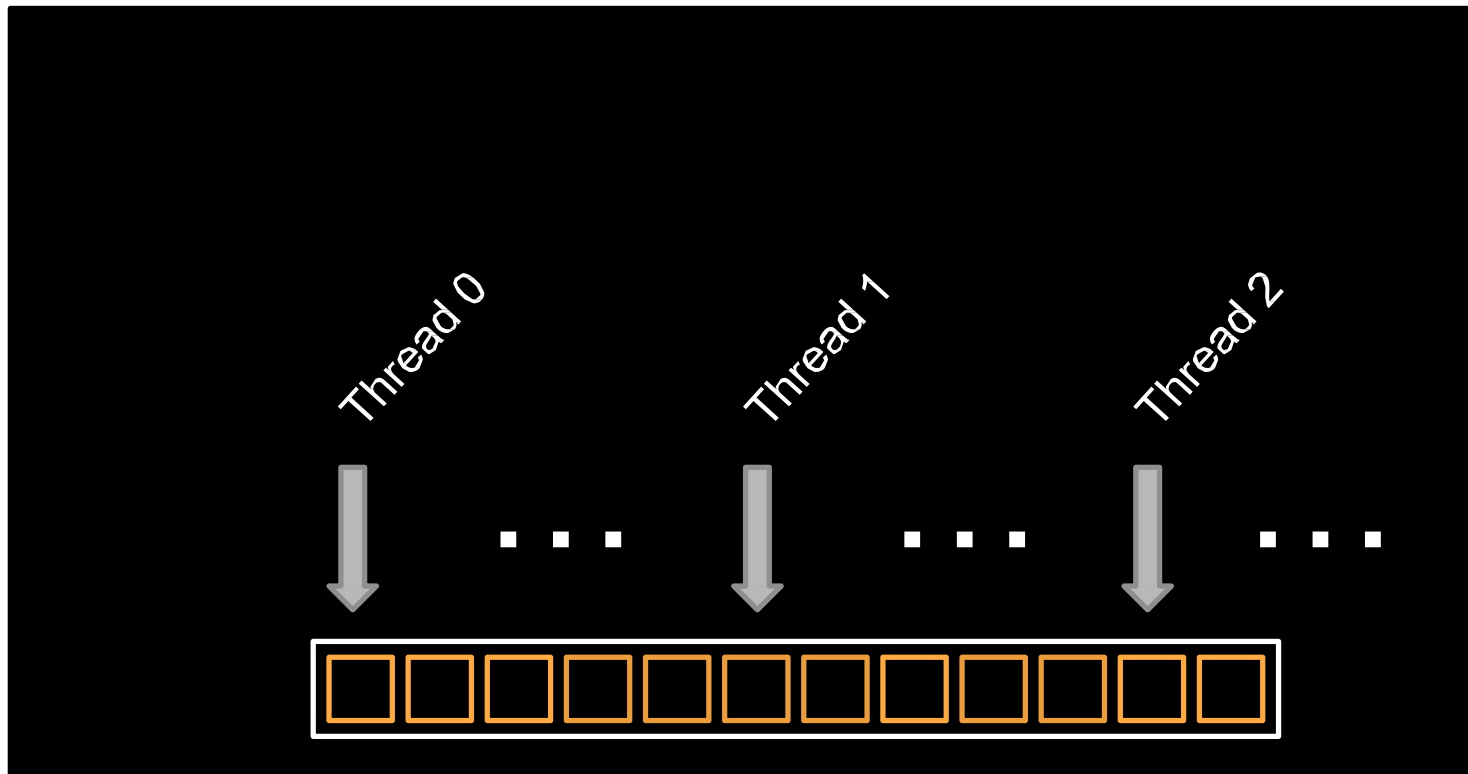
- **misaligned** memory access

chunk通常以32/64/128字节进行对齐



memory coalescing

- **uncoalesced** memory access





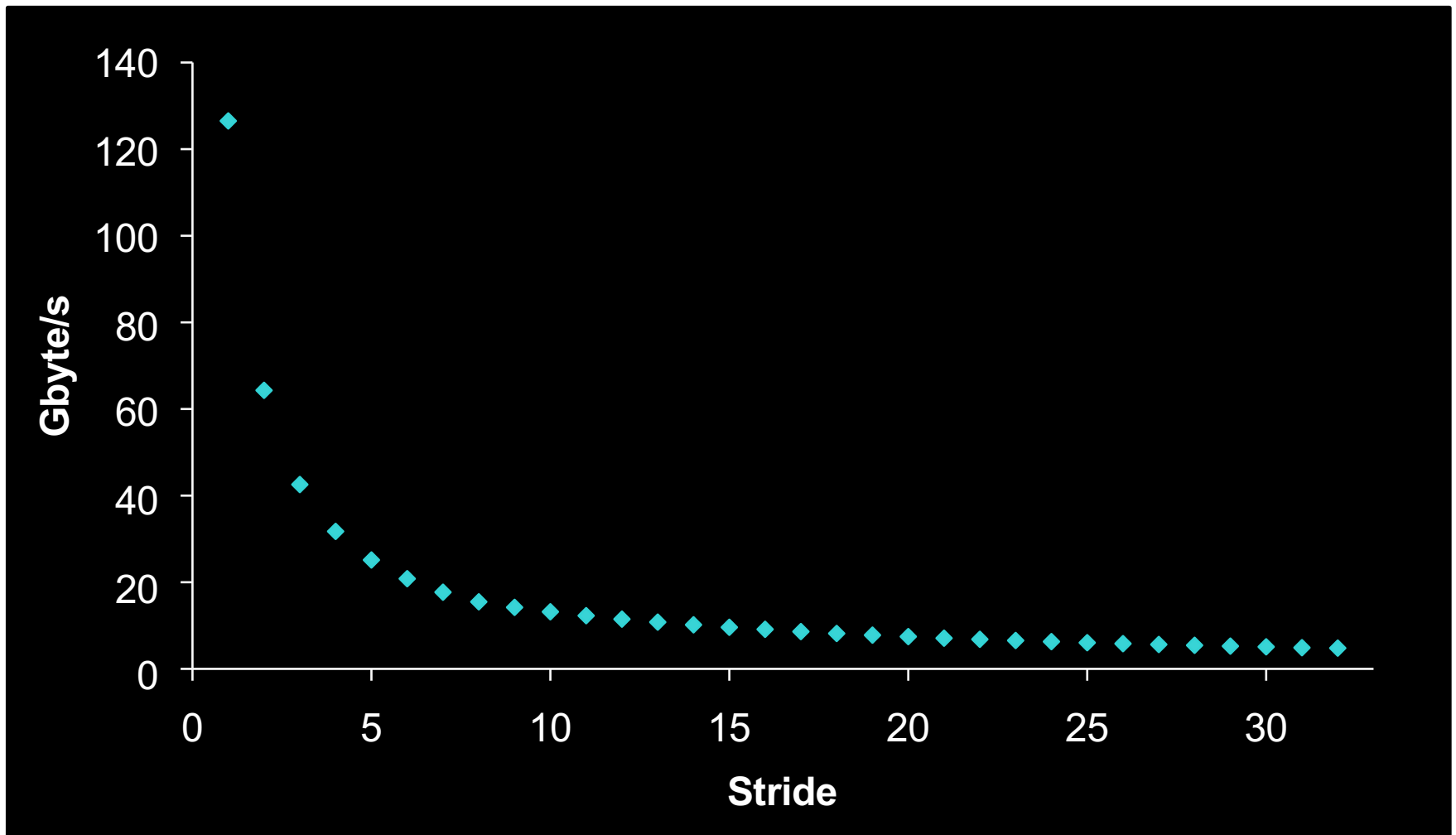
memory coalescing例子:

SAXPY

- SAXPY (Scalar Alpha X Plus Y) 是一个在 Basic Linear Algebra Subprograms (BLAS) 数据包中的函数 $y[] = a * x[] + y[]$
- 例子: SAXPY with stride
- 当步长为1时访问模式为fully coalesced

```
for (int i = 0; i < N; i++)  
    y[stride * i] += a * x[stride * i];
```

memory coalescing例子: SAXPY





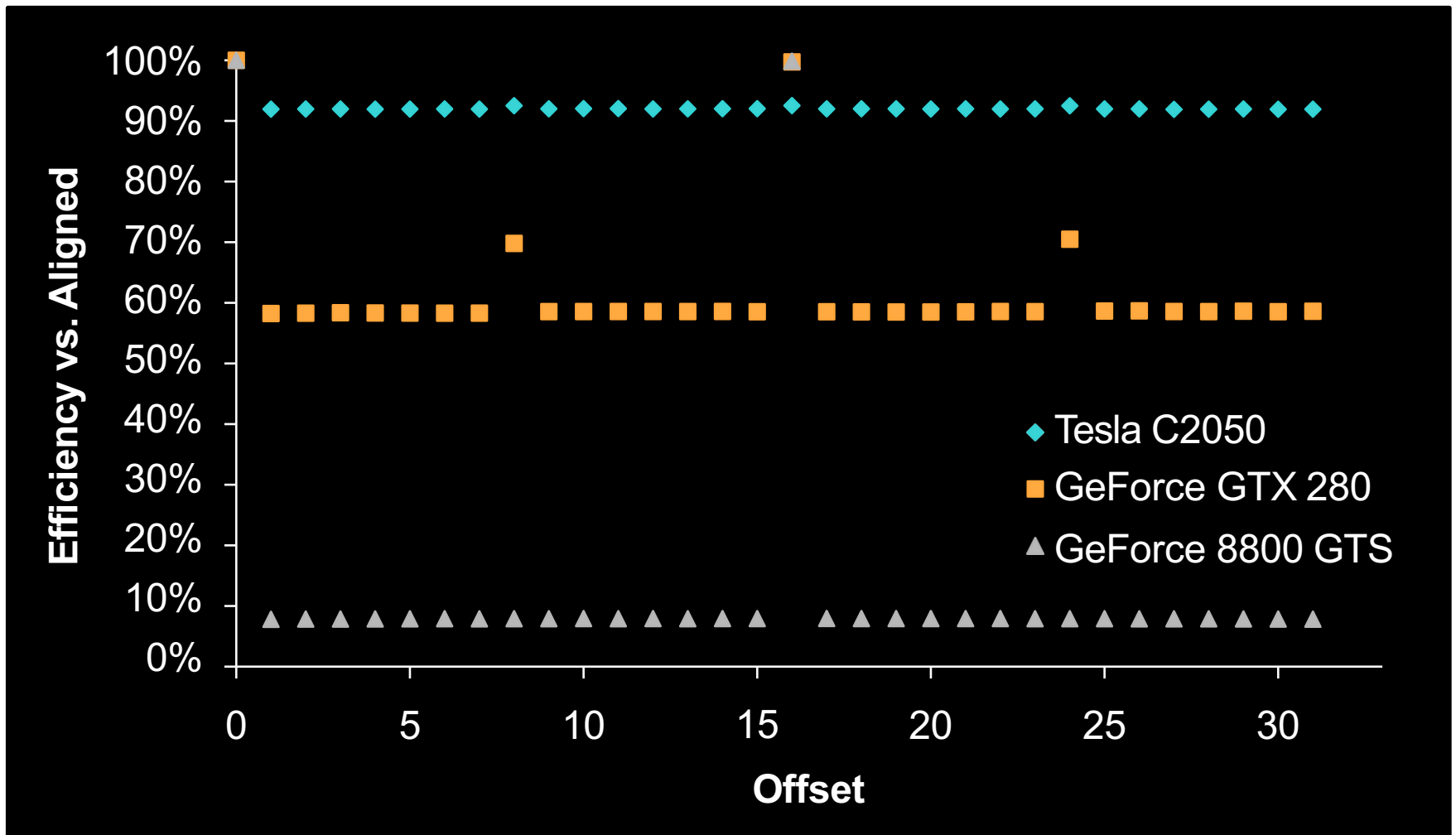
memory coalescing例子: SAXPY

- 例子: SAXPY with offset
- 当offset为0时为aligned

```
for (int i = 0; i < N; i++)  
    y[i + offset] += a * x[i + offset];
```



内存对齐的影响 (SAXPY)





思考

- memory coalescing指什么？



SpMV的内存访问

- 读取矩阵数据结构
 - 由具体的矩阵格式决定
 - 通常占用大量内存带宽
- 读取输入向量(x)
 - 由矩阵所决定
 - 很难对访问进行控制
 - 存在重用的可能性
- 向结果向量写数据(y)
 - 几乎不占用内存带宽

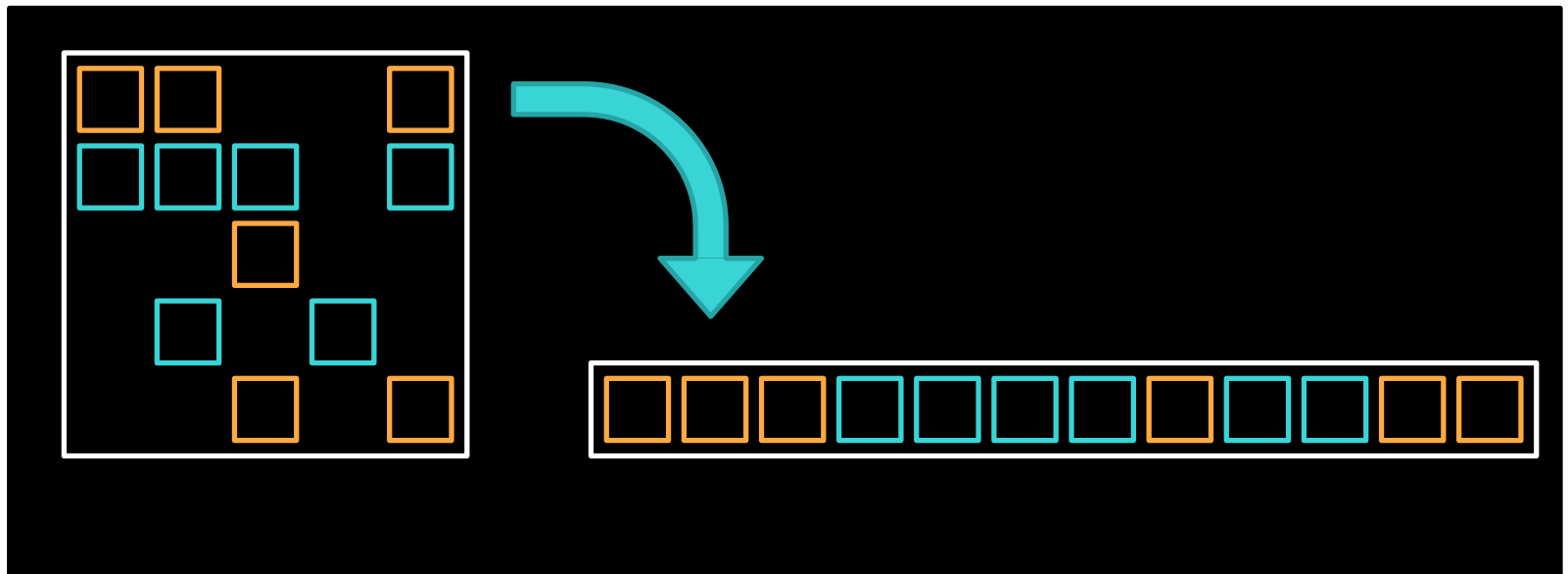
Compressed Sparse Row(CSR)

- 数据按行存储
- 不利于细粒度并行

4.3.3 CSR Format

RowPtr	=	[0 2 4 7 9]
Col	=	[0 1, 1 2, 0 2 3, 1 3]
Data	=	[3 7, 4 8, 1 5 9, 2 6]

Figure 5: The CSR format of matrix B.





CSR SpMV串行版本

```
void csr_spmv(int      num_rows,
              int      * row_offsets,
              int      * column_indices,
              float * values,
              float * x,
              float * y)
{
    for (int row = 0; row < num_rows; row++)
    {
        int row_begin = row_offsets[row];
        int row_end   = row_offsets[row + 1];

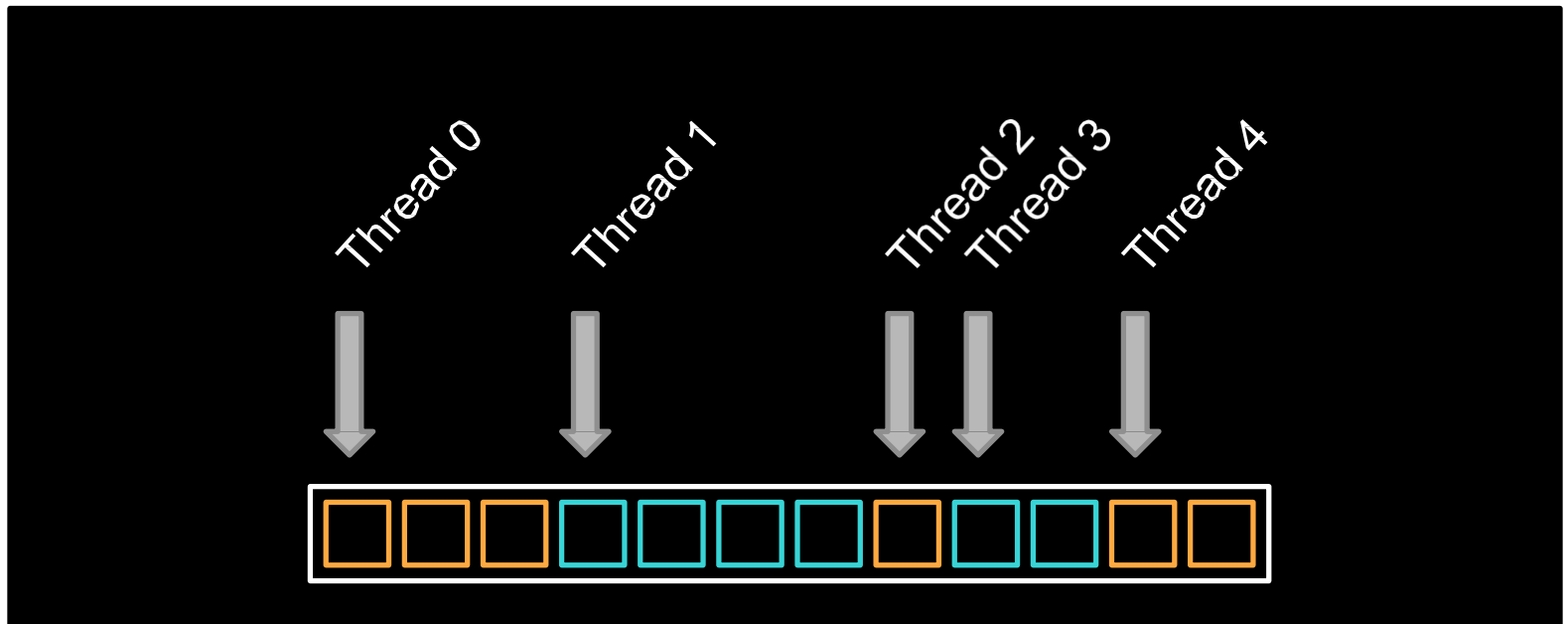
        float sum = 0;

        for (int offset = row_begin; offset < row_end; offset++)
            sum += values[offset] * x[column_indices[offset]];

        y[row] = sum;
    }
}
```

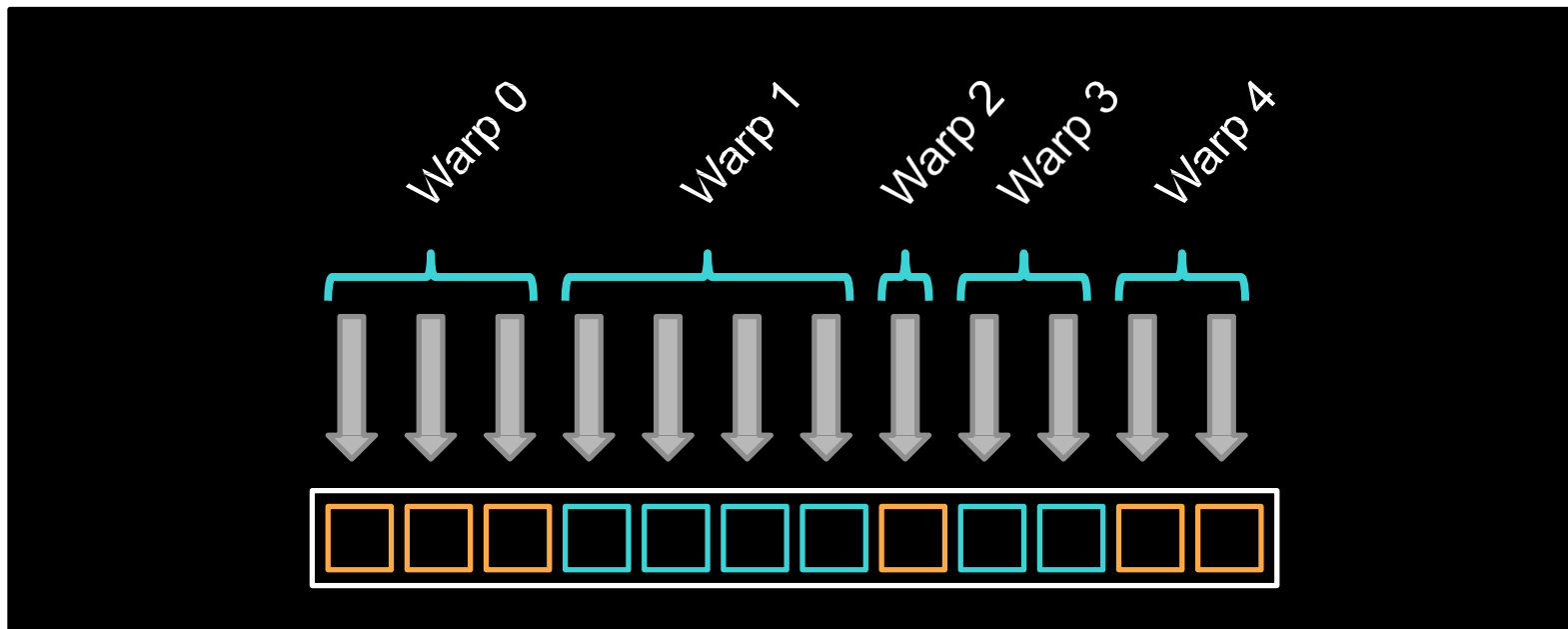
CSR(scalar) kernel

- 每个线程处理一行
 - memory uncoalescing
 - unaligned memory access(非对齐)



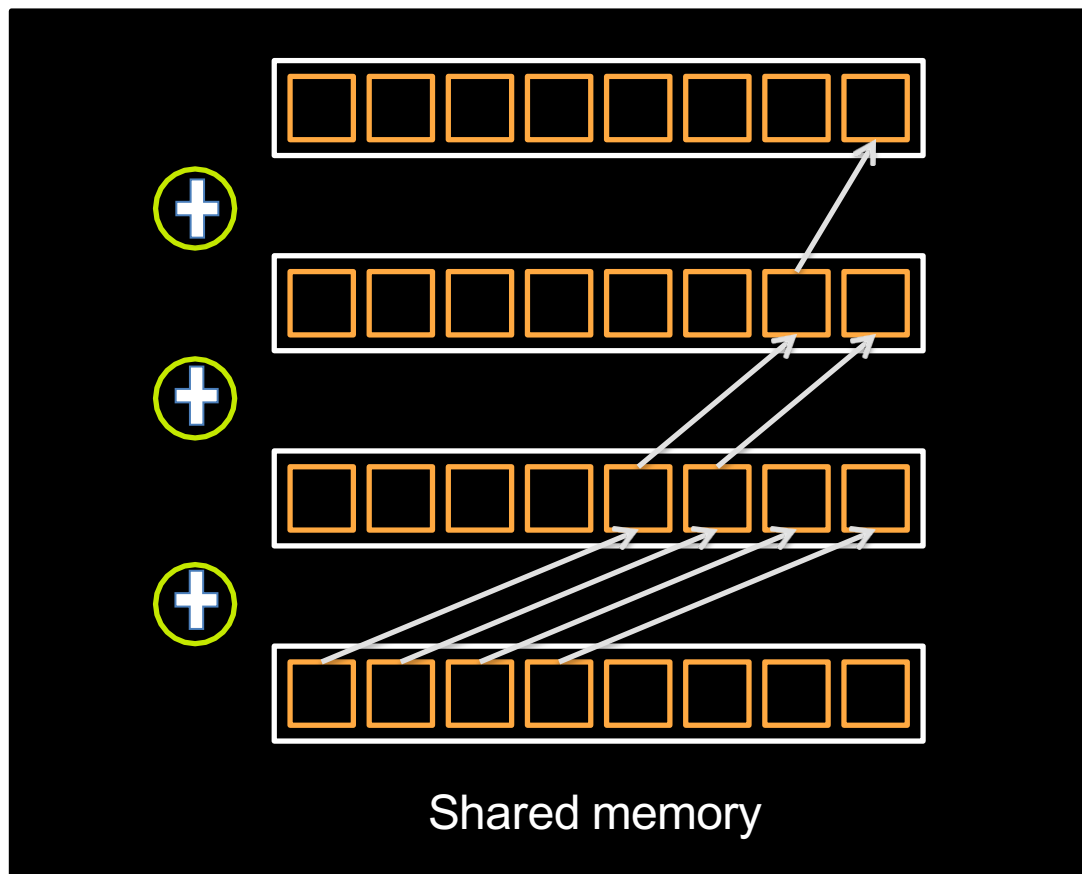
CSR(vector) kernel

- 每个SIMD vector/warp处理一行
 - partial memory coalescing
 - unaligned memory access



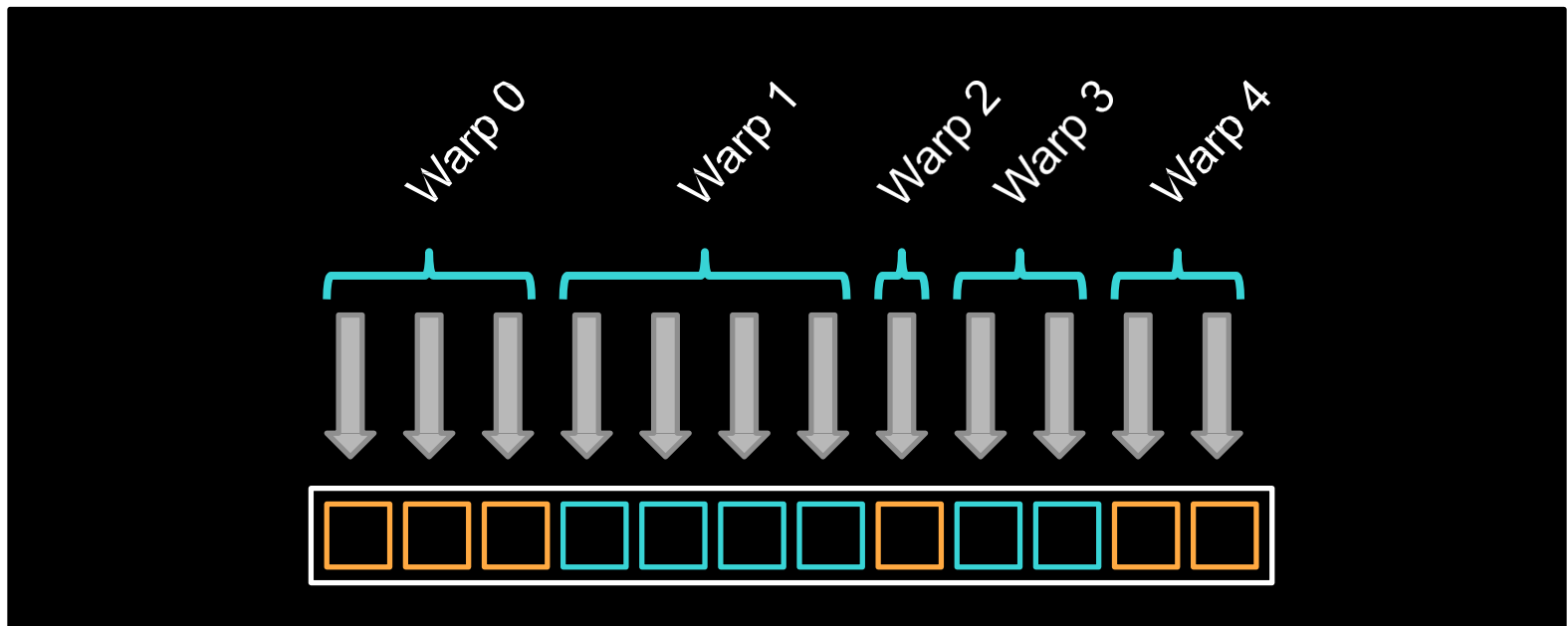
CSR(vector) kernel

- 利用shared memory局部求和
 - 假设warp只有8个线程



CSR(vector) kernel

- 每个SIMD vector/warp处理一行
 - partial memory coalescing
 - unaligned memory access



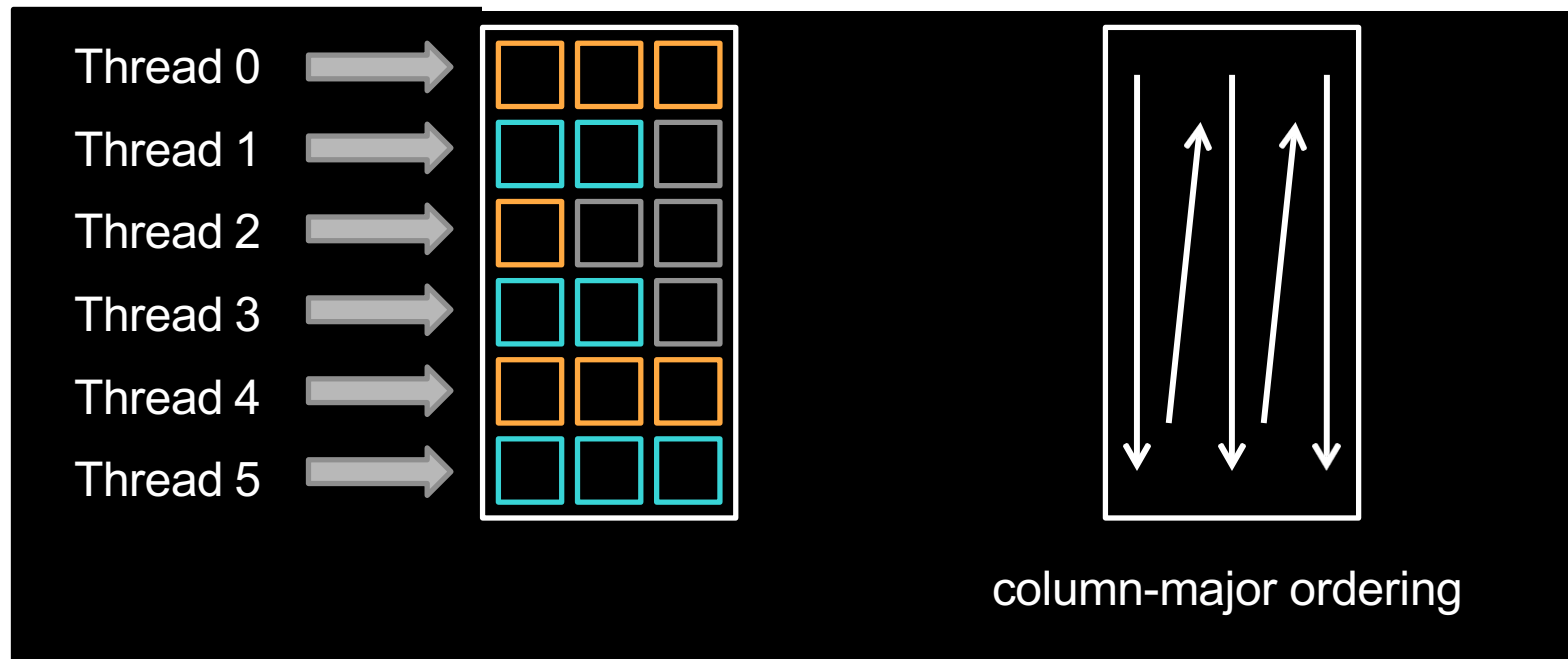
ELL kernel

$$B = \begin{bmatrix} 3 & 7 & 0 & 0 \\ 0 & 4 & 8 & 0 \\ 1 & 0 & 5 & 9 \\ 0 & 2 & 0 & 6 \end{bmatrix}$$

- full coalescing

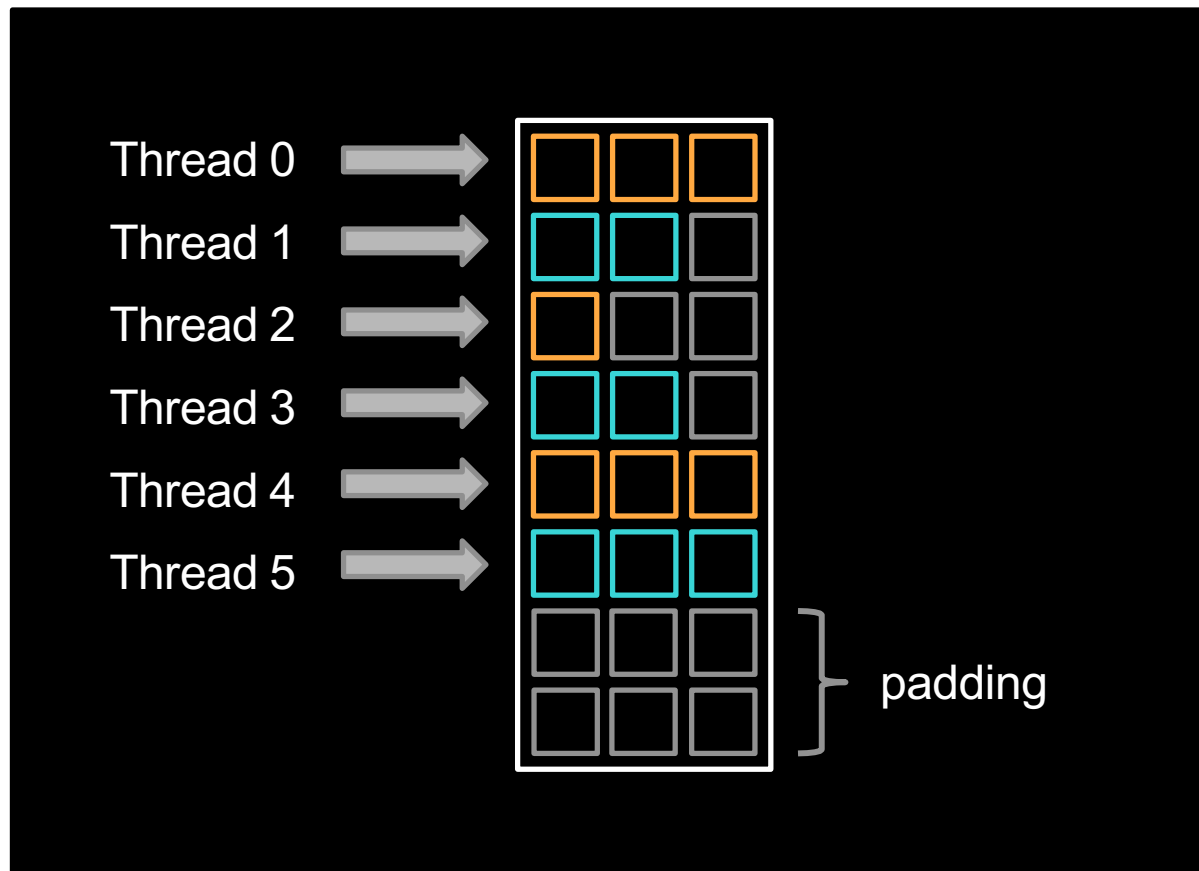
Col	=	[0 1 0 1, 1 2 2 3, 0 0 3 0]
Data	=	[3 4 1 2, 7 8 5 6, 0 0 9 0]

Figure 3: The ELL format of matrix B.



ELL kernel

- Pad columns for alignment



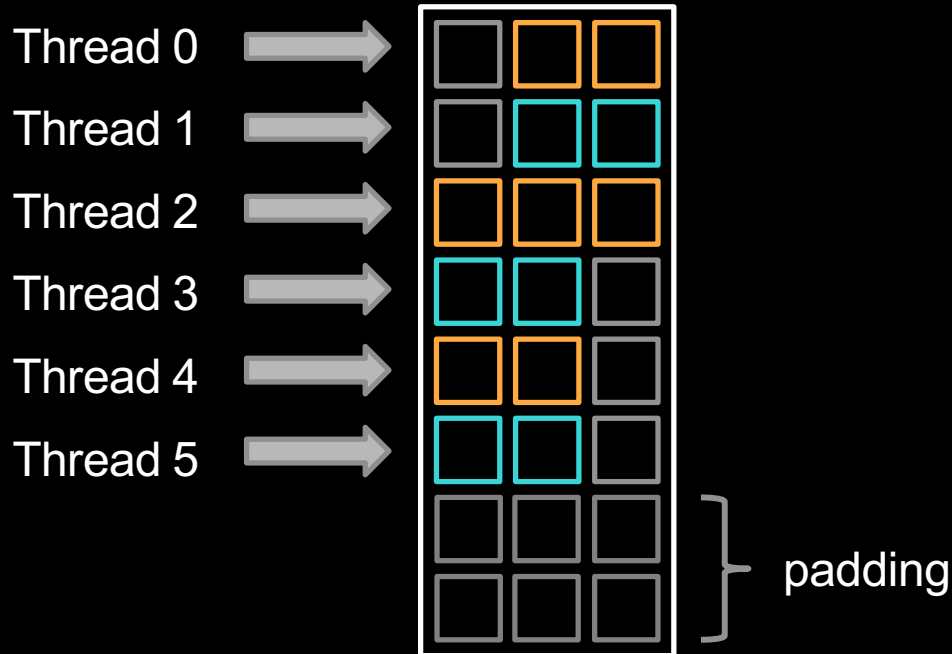
DIA kernel

$$B = \begin{bmatrix} 3 & 7 & 0 & 0 \\ 0 & 4 & 8 & 0 \\ 1 & 0 & 5 & 9 \\ 0 & 2 & 0 & 6 \end{bmatrix}$$

- 和ELL格式好处一样

Offsets	=	$[-2 \ 0 \ 1]$
Data	=	$[0 \ 0 \ 1 \ 2, \ 3 \ 4 \ 5 \ 6, \ 7 \ 8 \ 9 \ 0]$

Figure 1: The DIA format of matrix B.



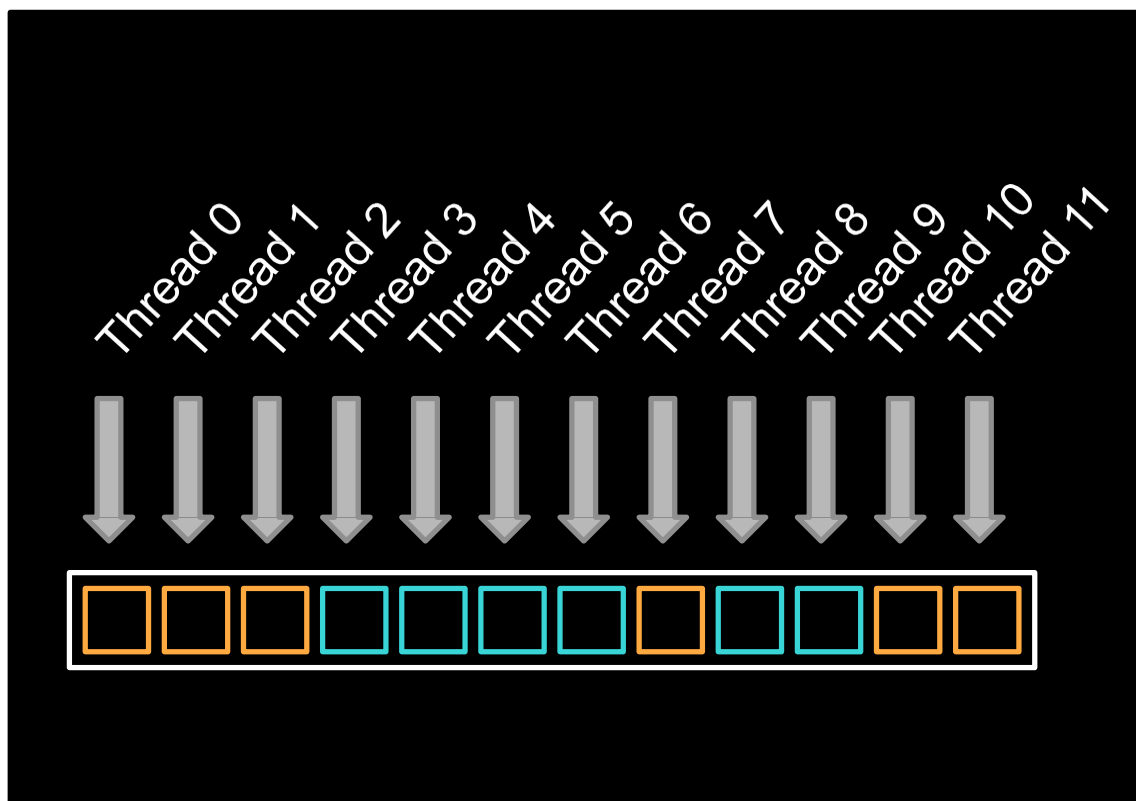
COO kernel

4.3.4 COO Format

- 每个线程处理一个非零元
— fully coalesced

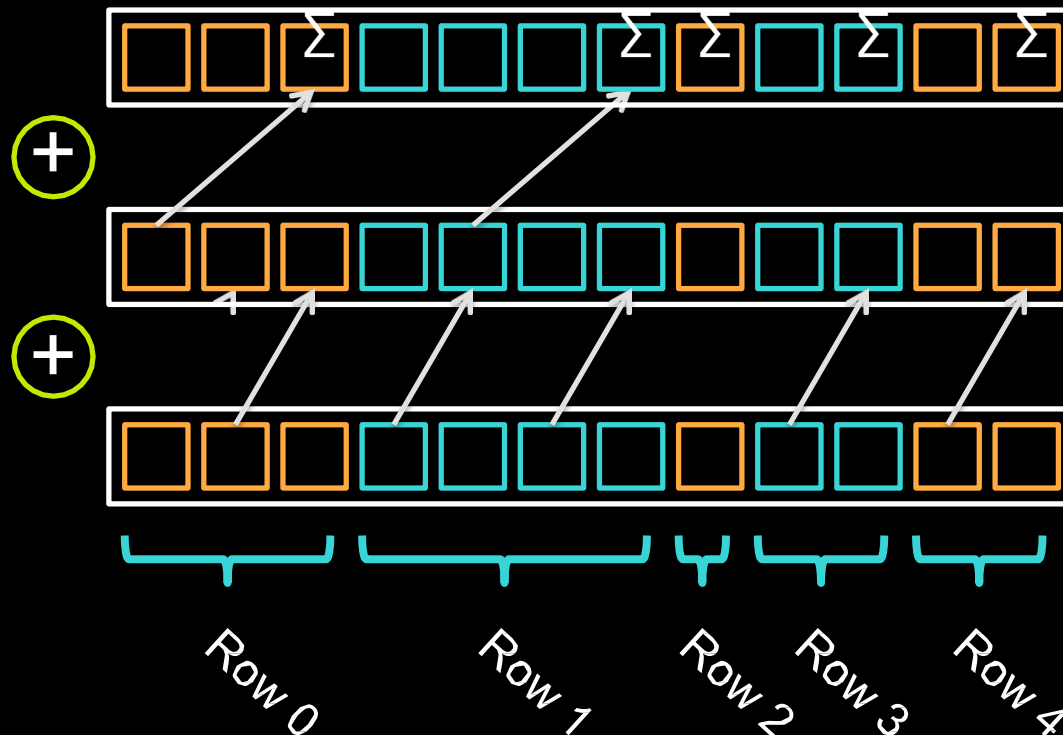
Row	=	[0 0, 1 1, 2 2 2, 3 3]
Col	=	[0 1, 1 2, 0 2 3, 1 3]
Data	=	[3 7, 4 8, 1 5 9, 2 6]

Figure 6: The COO format of matrix B.



COO kernel

- Store i and $A(i,j) * x(j)$ in shared memory
 - Compute row sums using segmented reduction





memory coalescing部分 算法总结

Full Coalescing

DIA, ELL, and COO

Partial Coalescing

CSR (vector)

Efficiency depends on row length

Little Coalescing

CSR (scalar)



思考

- 高效利用内存带宽，SpMV中是如何运用的？

性能考虑因素

- 高效利用内存带宽
 - memory coalescing
- 细粒度并行
 - 数以千计的线程
- 尽可能多的数据重用
 - 利用缓存、shared memory





细粒度并行

DIA, ELL & CSR (scalar)

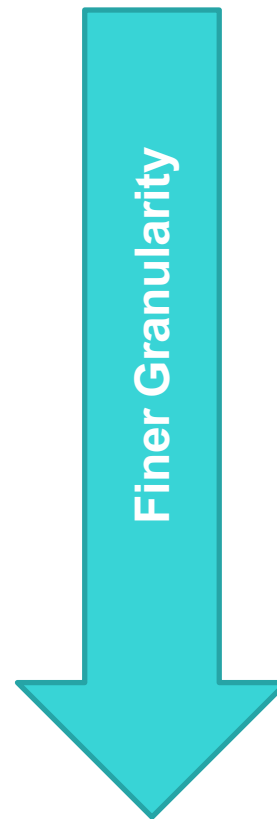
每个线程处理一行

CSR (vector)

每个warp处理一行

COO

每个线程处理一个非零元



$2K \times 2K$

$512 \times 8K$

$2 \times 2M$

$1 \times 4M$

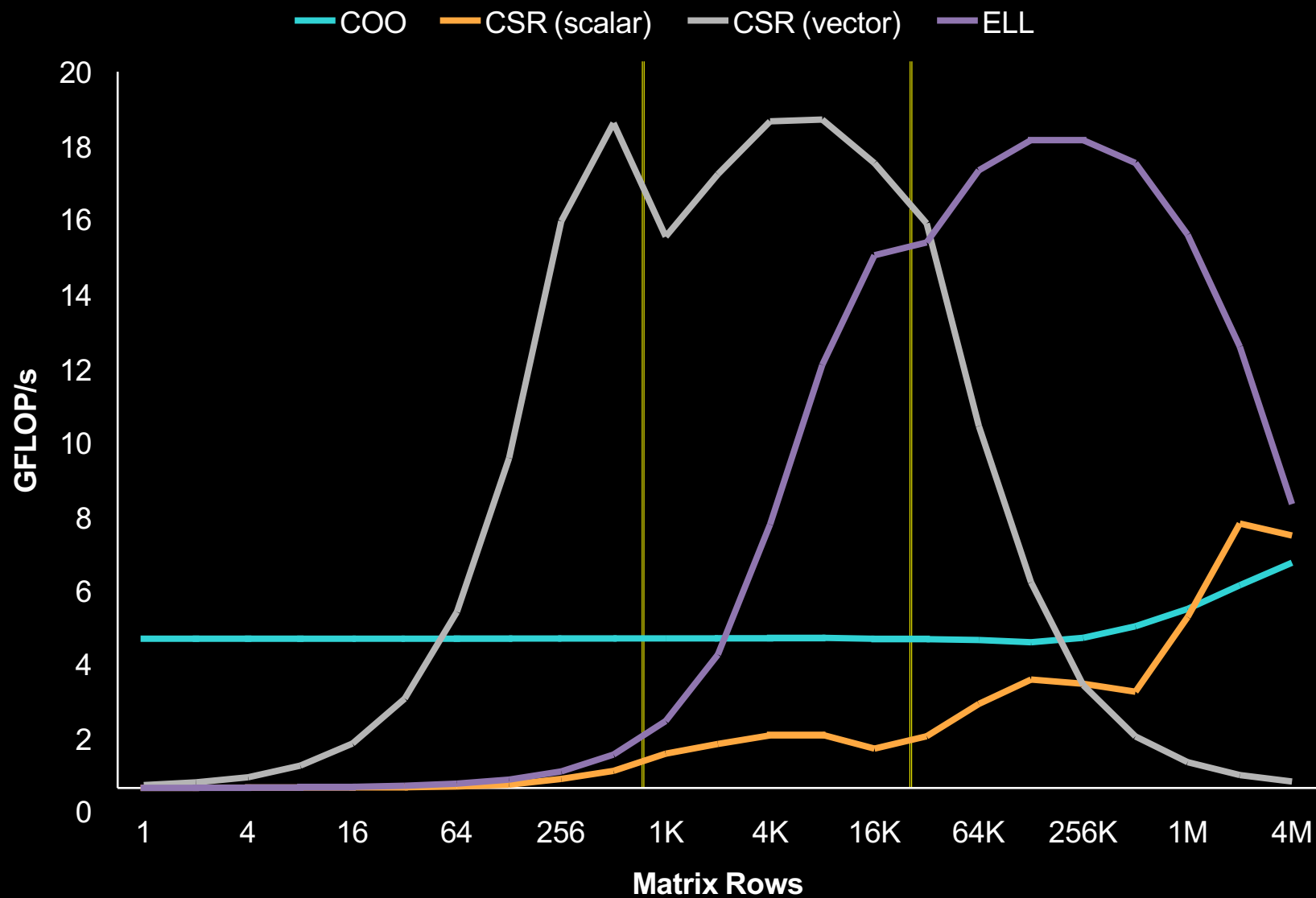
$1K \times 4K$

$4K \times 1K$

$8K \times 512$

$2M \times 2$

所有矩阵都有4M个非零元素



GPU: GeForce GTX 285



细粒度并行部分 算法总结

每个线程处理一行

ELL, DIA, and CSR (scalar) kernel

适合处理行数多的矩阵

每个 **warp** 处理一行

CSR (vector) kernel

适合处理行数少的矩阵 (>256)

每个线程处理一个元素

COO kernel

对矩阵形状不敏感



思考

- 如何利用不同格式的特点进行稀疏矩阵运算？



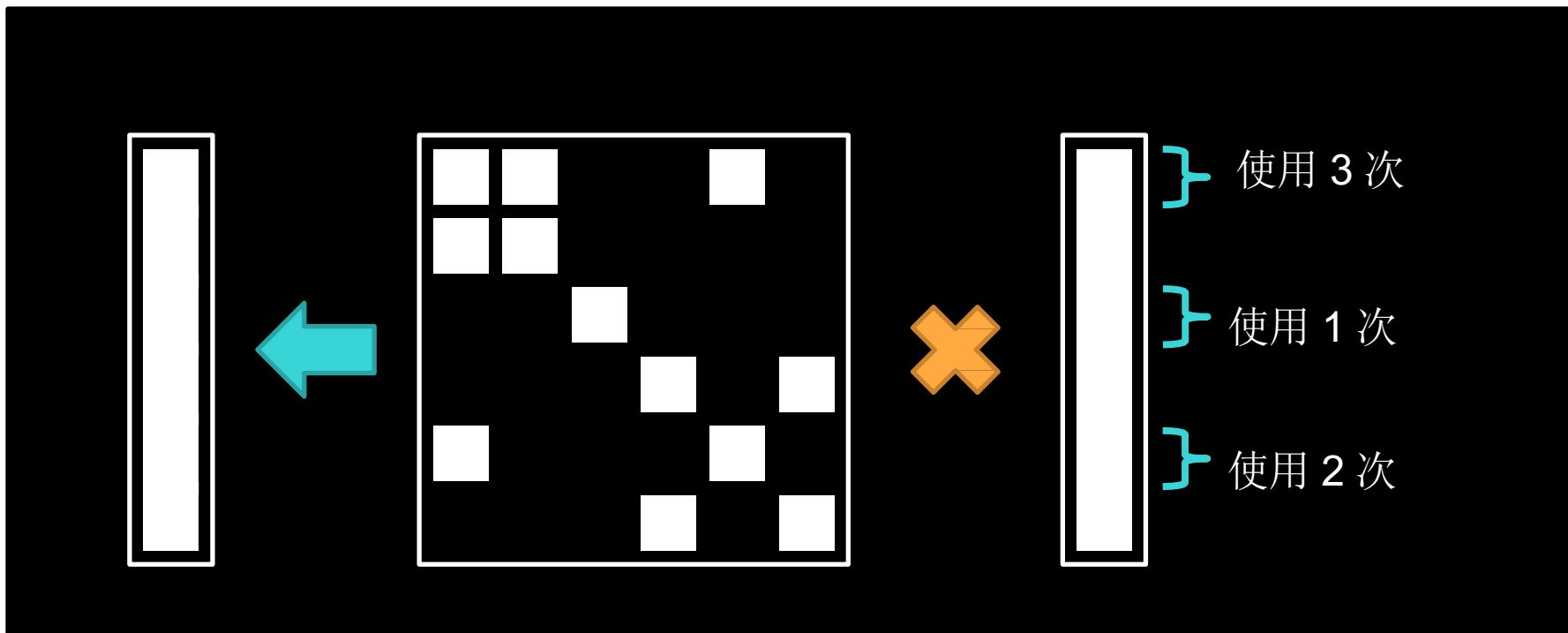
性能考虑因素

- 高效利用内存带宽
 - memory coalescing
- 细粒度并行
 - 数以千计的线程
- 尽可能多的数据重用
 - 利用缓存、shared memory



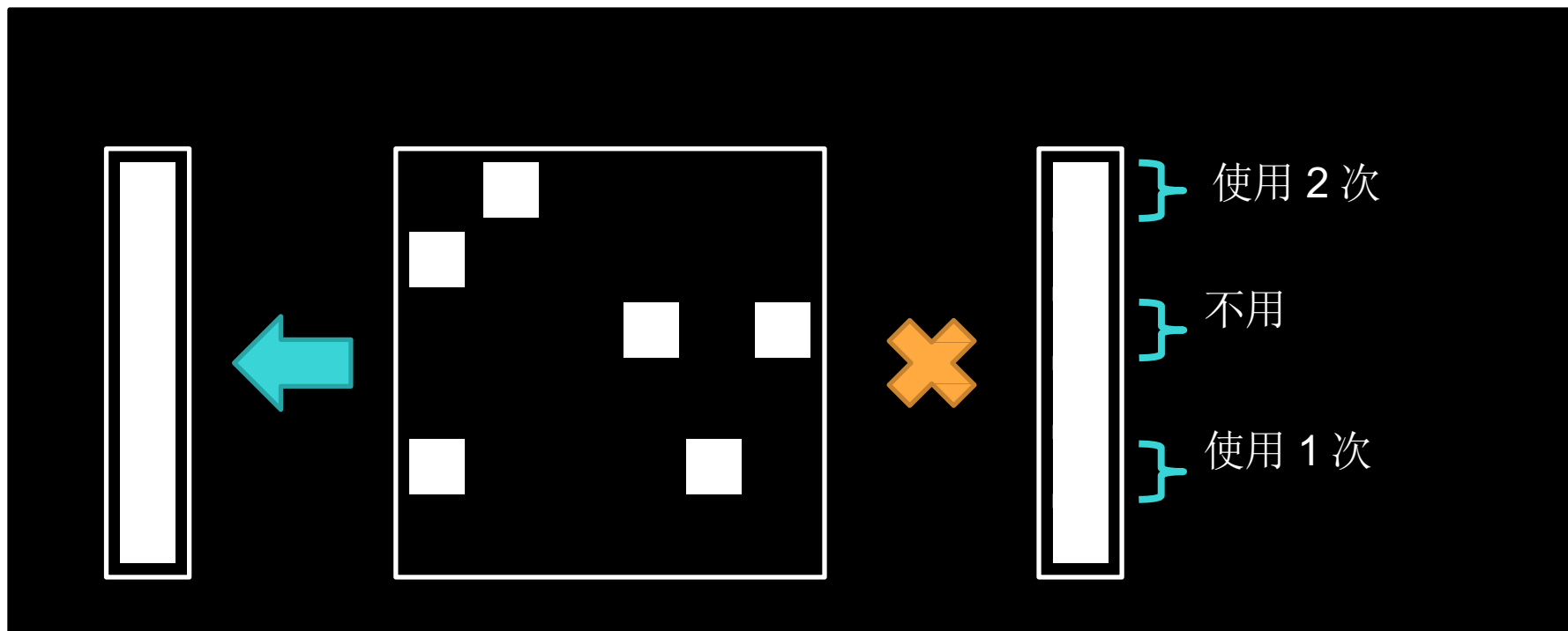
caching

- 重复访问列向量



caching

- 效率取决于输入矩阵的非零元分布





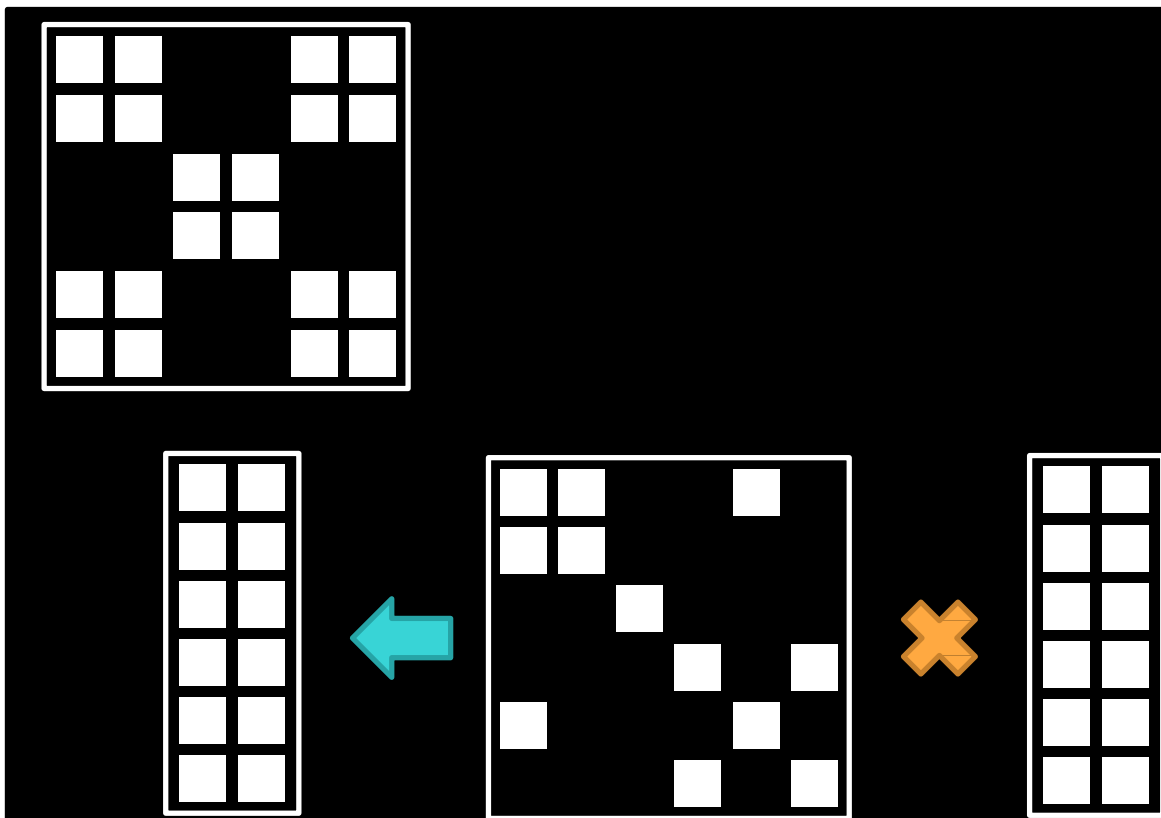
caching

- 帶有L1 cache的GPU
 - 不需要任何操作
- 帶有texture cache的GPU
 - 性能提升30%左右
- 软件控制的cache
 - 提前放到shared memory中
- 效率取决于矩阵分布

其他技术

- blocking
 - 降低编号开销(indexing)

- 多个列向量
 - 矩阵重用





性能考虑因素

- 高效利用内存带宽
 - memory coalescing
- 细粒度并行
 - 数以千计的线程
- 尽可能多的数据重用
 - 利用缓存、shared memory





A Synchronization-Free Algorithm for Parallel Sparse Triangular Solves

Weifeng Liu (University of Copenhagen, Denmark, &
STFC Rutherford Appleton Laboratory, UK)

Ang Li (Eindhoven University of Technology, Netherlands)

Jonathan D. Hogg (STFC Rutherford Appleton Laboratory, UK)

Iain S. Duff (STFC Rutherford Appleton Laboratory, UK)

Brian Vinter (University of Copenhagen, Denmark)

论文见obe

Euro-Par '16, Grenoble, France, 25 August 2016



Sparse Matrix & Sparse Triangular Solve (SpTRSV)

Sparse Matrix (in the CSR format)

- 如果矩阵中绝大多数的元素为0，那么可以考虑用稀疏矩阵的存储格式只存储其中的非零元素（**nonzero**）
- The **CSR** format stores a matrix **row**-wise

0	0	1	0
2	3	0	0
0	0	0	0
4	0	5	6

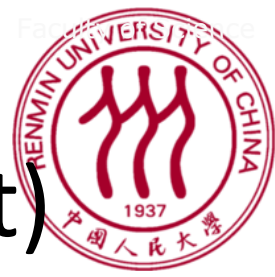
A
(4x4)
dense

		1	
2	3		
4		5	6

A
(4x4)
 $nnzA = 6$

row pointer =	0	1	3	3	6	
column index =	2	0	1	0	2	3
value =	1	2	3	4	5	6

A (in the CSR format)
(4x4)
 $nnzA = 6$



Sparse Matrix (in the CSC format)

- 如果矩阵中绝大多数的元素为0，那么可以考虑用稀疏矩阵的存储格式只存储其中的非零元素（nonzero）
- The **CSC** format stores a matrix **column**-wise.

0	0	1	0
2	3	0	0
0	0	0	0
4	0	5	6

A
(4x4)
dense

		1	
2	3		
4		5	6

A
(4x4)
 $nnzA = 6$

column pointer =	0	2	3	5	6	
row index =	1	3	1	0	3	3
value =	2	4	3	1	5	6

A (in the CSC format)
(4x4)
 $nnzA = 6$

Hao Wang, Weifeng Liu, Kaixi Hou, and Wu-Chun Feng. **Parallel Transposition of Sparse Data Structures**. *ACM International Conference on Supercomputing '16*.

Sparse Triangular Solve

- Example: $Lx = b$

计算向量 x ， L 为稀疏下三角矩阵， b 为运行时可知的向量

1			
	1		
	2	1	
3			1

L
(4x4)
 $nnzL = 6$
known

x

x_0
x_1
x_2
x_3

x
(4x1)
dense
unknown

=

a
b
c
d

b
(4x1)
dense
known

system:



solution:

$$\begin{cases} 1 * x_0 = a \\ 1 * x_1 = b \\ 2 * x_1 + 1 * x_2 = c \\ 3 * x_0 + 1 * x_3 = d \end{cases}$$

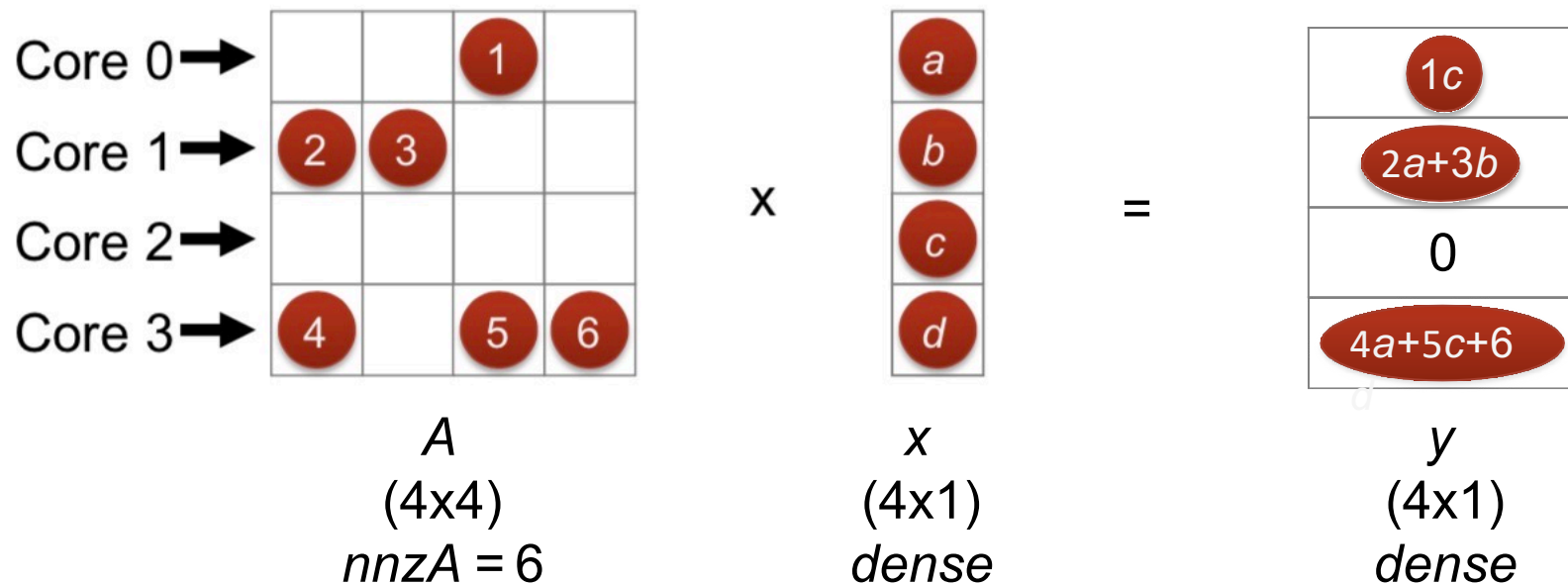
$$\begin{cases} x_0 = a \\ x_1 = b \\ x_2 = c - 2b \\ x_3 = d - 3a \end{cases}$$



Why parallelizing SpTRSV is not trivial?
为什么SpTRSV复杂?

Sparse Matrix-Vector Multiplication

- 稀疏矩阵向量乘：稀疏矩阵 A 和稠密向量 x 相乘获得向量 y

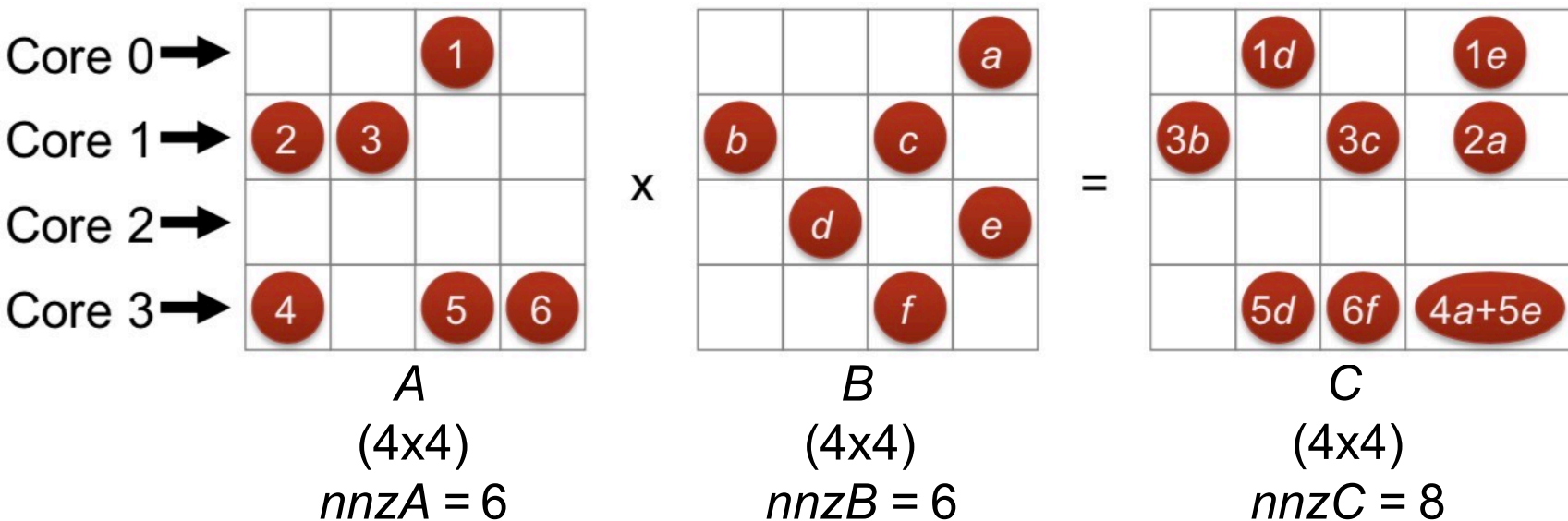


Weifeng Liu and Brian Vinter. **CSR5: An Efficient Storage Format for Cross-Platform Sparse Matrix-Vector Multiplication.** *ACM International Conference on Supercomputing '15.*

Sparse Matrix-Matrix Multiplication

- Example: $C = AB$

稀疏矩阵-稀疏矩阵乘：稀疏矩阵 A 和稀疏矩阵 B 相乘
获得稀疏矩阵 C 。

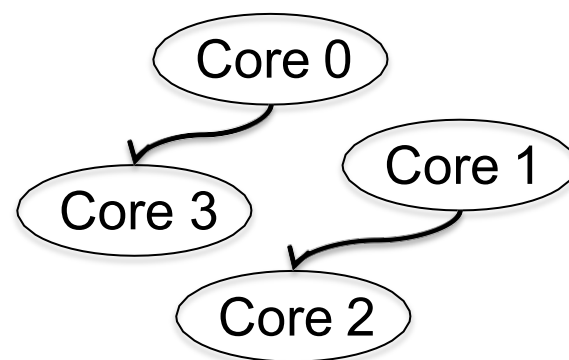
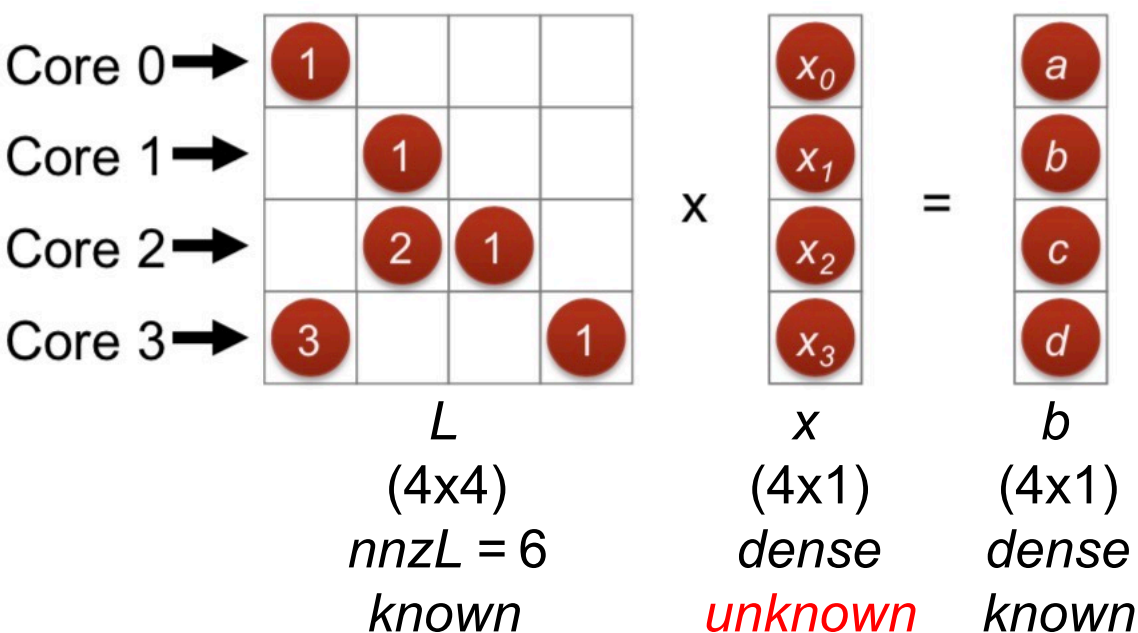


Weifeng Liu and Brian Vinter. **A Framework for General Sparse Matrix-Matrix Multiplication on GPUs and Heterogeneous Processors.**
Journal of Parallel and Distributed Computing '15.

Sparse Triangular Solve

- Example: $Lx = b$

SpTRSV存在内部的串行化依赖关系。在下图的例子中，core 2 需要等待core 1的计算结果， core 3需要等待core 0的计算结果，这也意味着4个core无法并行进行计算。



$$\left\{ \begin{array}{l} 1 * x_0 = a \\ 1 * x_1 = b \\ 2 * x_1 + 1 * x_2 = c \\ \underline{3 * x_0 + 1 * x_3 = d} \end{array} \right.$$



The level-set method (related work)

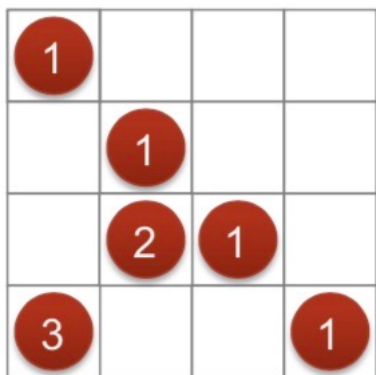
Anderson and Saad [1] and Saltz [23]

[1] Anderson, E., Saad, Y.: Solving Sparse Triangular Linear Systems on Parallel Computers. *International Journal of High Speed Computing* 1(1), 73–95 (1989)

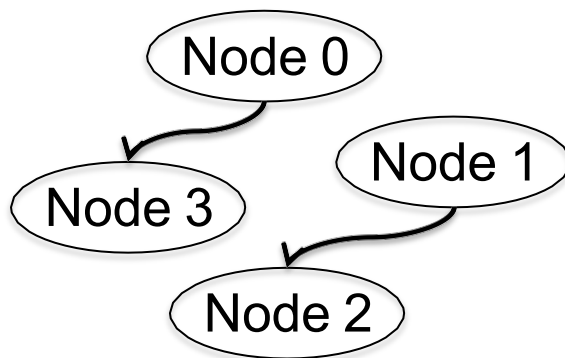
[23] Saltz, J.H.: Aggregation Methods for Solving Sparse Triangular Systems on Multiprocessors. *SIAM Journal on Scientific and Statistical Computing* 11(1), 123–144 (1990)

Level-set method for parallel SpTRSV

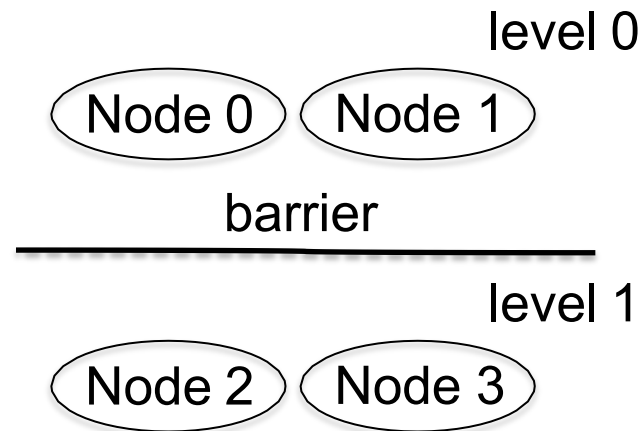
- level-set方法分为2个阶段：
 - (1) 将能够并行处理的点（行或列）聚为一个集合set
 - (2) 之后一组一组分层处理，各层间有barrier



matrix form



graph form



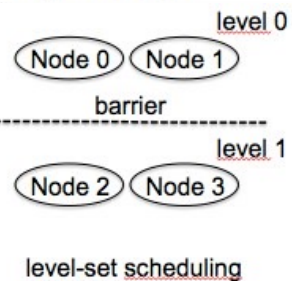
level-set scheduling

Level-set method for parallel SpTRSV



- 然而，level-set的方法1) 预处理时间（分组），2) 运行时显式barrier同步时间开销过大。

Matrix name	Preprocessing cost (ms)	SpTRSV cost (ms)	SpTRSV cost breakdown (ms)		#Level-sets
			Synchronization	Compute	
FEM/ship-003	92.46	12.95	10.96	1.99	4367
FEM/Cantilever	47.89	9.60	5.62	3.98	2397
chipcool0	8.74	1.99	1.15	0.84	534
nlpkkt160	484.67	38.30	0.01	38.29	2



**Preprocessing overhead
is equal to cost for a few
SpTRSV operations**

**Synchronization
dominates
SpTRSV
when #level-sets
is high**

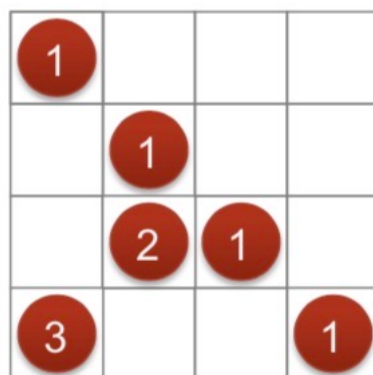


Synchronization-Free SpTRSV (this work)

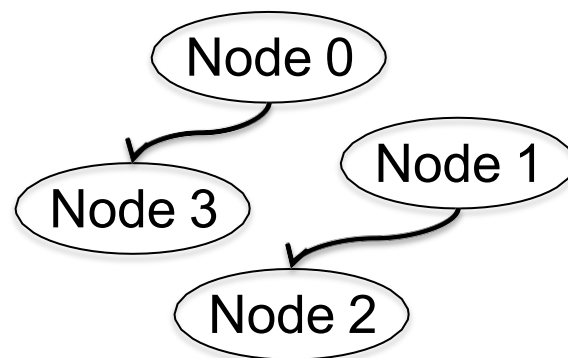
Basic idea: in-degree and spin-lock

- 所有的计算核心都保持活跃状态：其中一些核心计算没有依赖的部分，另一些忙等待（**busy-wait**），等待自旋锁 **spin-locks** 打开表示依赖解除可以进行后续计算。

$$\left\{ \begin{array}{l} 1 * x_0 = a \\ 1 * x_1 = b \\ 2 * x_1 + 1 * x_2 = c \\ 3 * x_0 + 1 * x_3 = d \end{array} \right.$$



matrix form



graph form

CSR row_ptr:	0	1	2	4	6
--------------	---	---	---	---	---

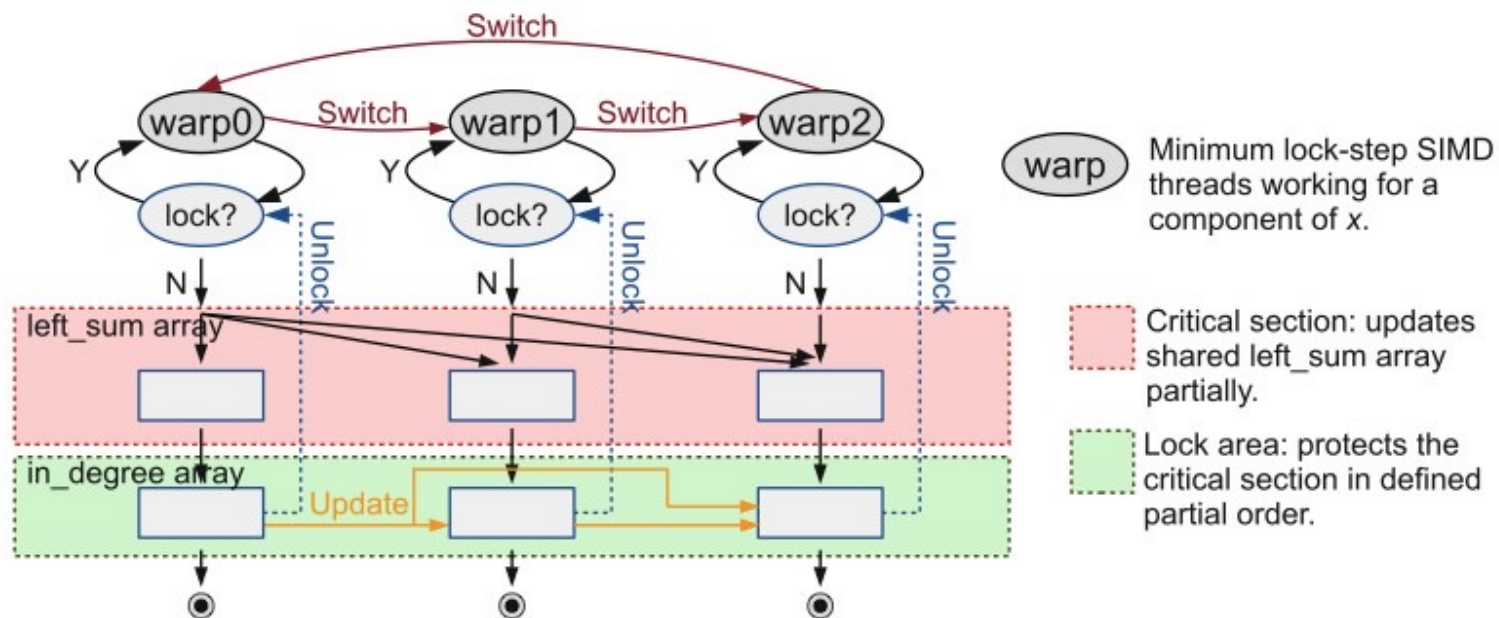
CSC column_ptr:	0	2	4	5	6
-----------------	---	---	---	---	---

In-degree:	1	1	2	2
------------	---	---	---	---

Out-degree:	2	2	1	1
-------------	---	---	---	---

Basic idea: warp/wavefront switch

- 每个warp负责求解一个 $x[i]$ ，不同的 $x[i]$ （warp间）可以分为block内通信和block间通信依赖，block内考虑使用shared memory，block间考虑使用global memory通信。



The proposed sync-free algorithm

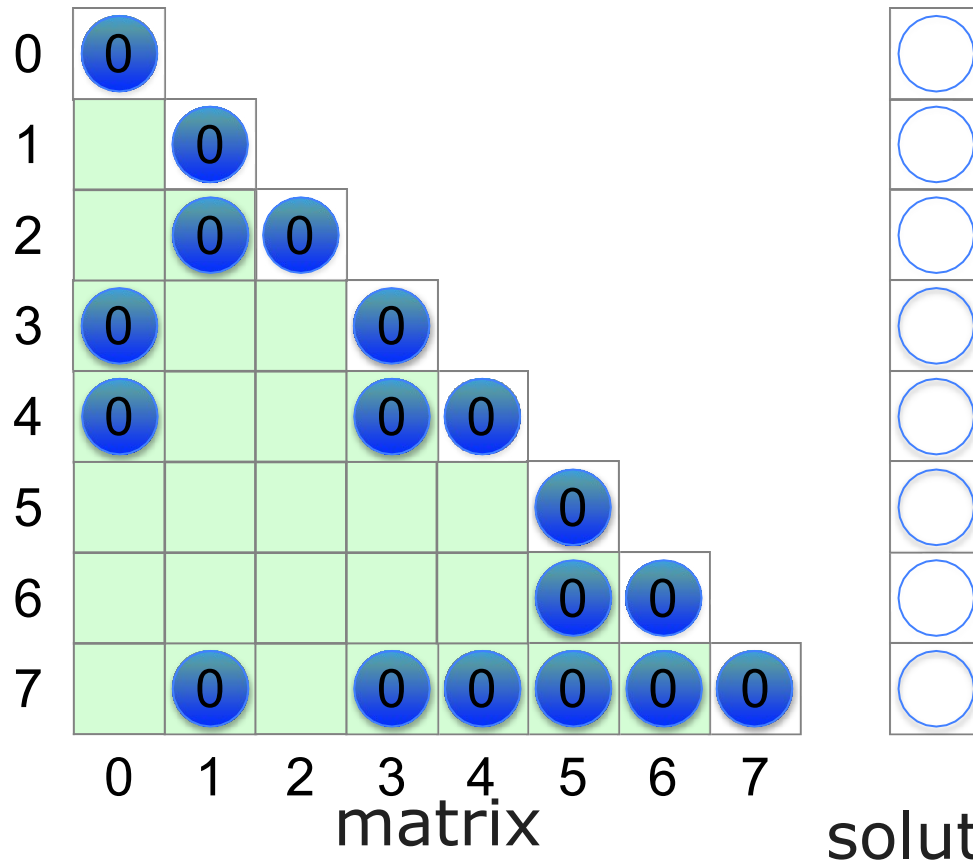
Algorithm 2. The proposed synchronization-free SpTRSV algorithm.

```

1: MALLOC(*d_left_sum, *s_left_sum, *d_in_degree, *s_in_degree, n)
2: MEMSET(*d_left_sum, *s_left_sum, *d_in_degree, *s_in_degree, 0)
3: function PREPROCESSING-STAGE()
4:   for  $i = 0$  to  $nnz - 1$  in parallel do
5:     ATOMIC-ADD(&d_in_degree[row_idx[i]], 1)
6:   end for
7: end function
8: function SOLVING-STAGE()
9:    $th \leftarrow \text{SET}()$  ▷ size of diagonal block
10:  for  $i = 0$  to  $n - 1$  in parallel do ▷ One concurrent warp for one component.
11:    while  $s\_in\_degree[i] + 1 \neq d\_in\_degree[i]$  do
12:      //busy wait
13:    end while
14:     $x[i] \leftarrow (b[i] - d\_left\_sum[i] - s\_left\_sum[i]) / val[col\_ptr[i]]$ 
15:    for  $j = col\_ptr[i] + 1$  to  $col\_ptr[i + 1] - 1$  in parallel do ▷ One thread for one nonzero.
16:       $rid \leftarrow row\_idx[j]$ 
17:      if  $rid < i + th - i \% th$  then ▷ Use on-chip scratchpad for red areas in Figure 3.
18:        ATOMIC-ADD(&s_left_sum[rid],  $val[j] \times x[i]$ )
19:        ATOMIC-ADD(&s_in_degree[rid], 1)
20:      else ▷ Use off-chip memory for green area in Figure 3.
21:        ATOMIC-ADD(&d_left_sum[rid],  $val[j] \times x[i]$ )
22:        ATOMIC-SUB(&d_in_degree[rid], 1)
23:      end if
24:    end for
25:  end for
26: end function
27: FREE(*d_left_sum, *s_left_sum, *d_in_degree, *s_in_degree)

```

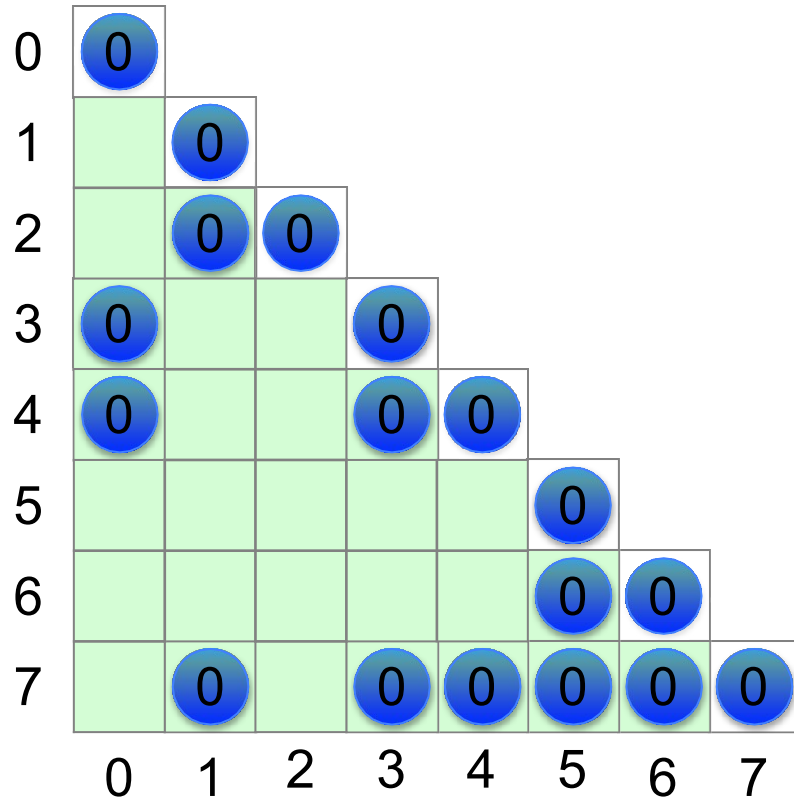
Cartoon of an example (init)



in-degree:

1	1	2	2	3	1	2	6
---	---	---	---	---	---	---	---

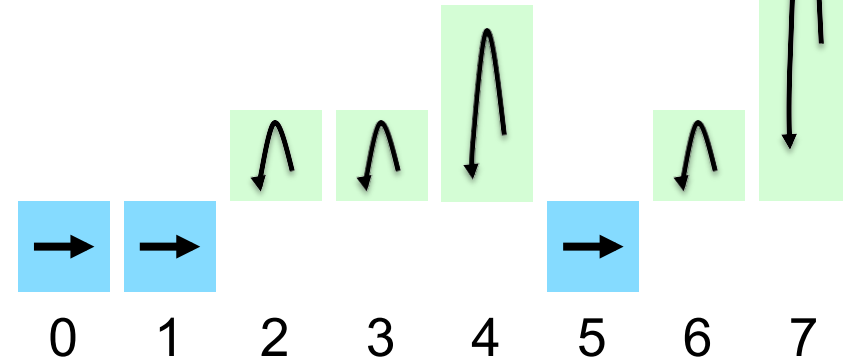
Cartoon of an example (step 1)



matrix



solution

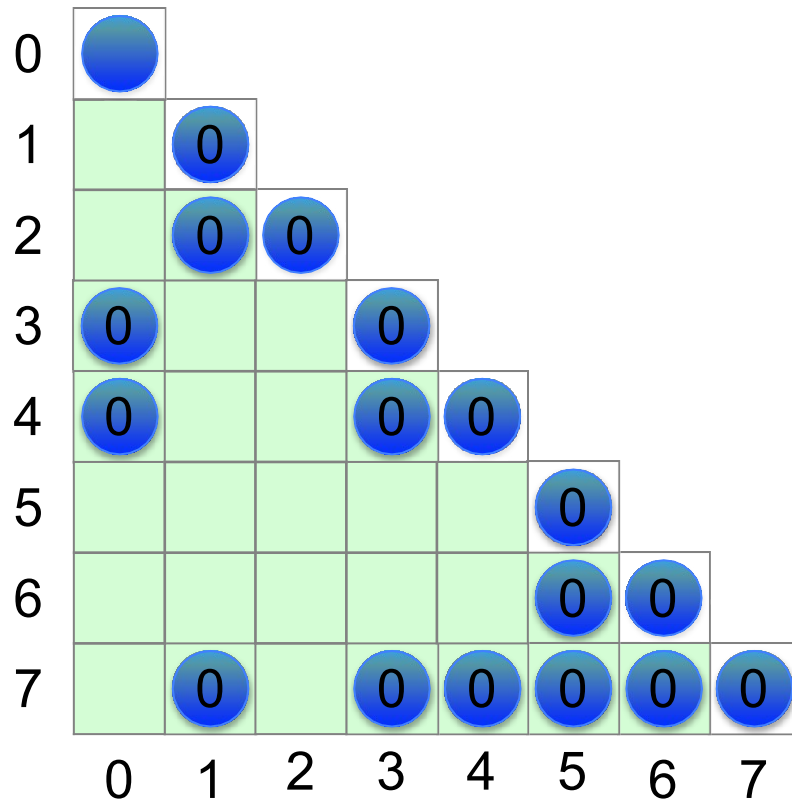


memory behavior

in-degree:

1	1	2	2	3	1	2	6
---	---	---	---	---	---	---	---

Cartoon of an example (step 2)

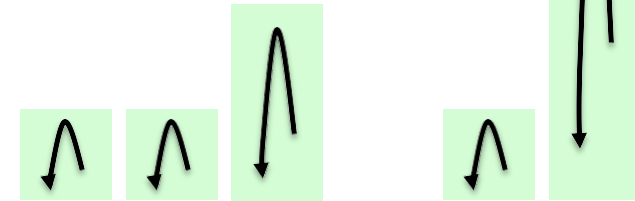


matrix



solution

0 1 2 3 4 5 6 7

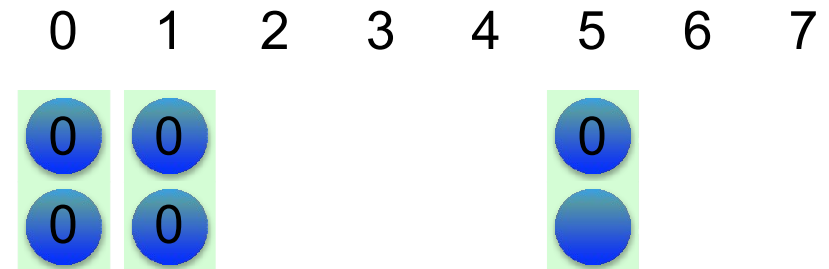
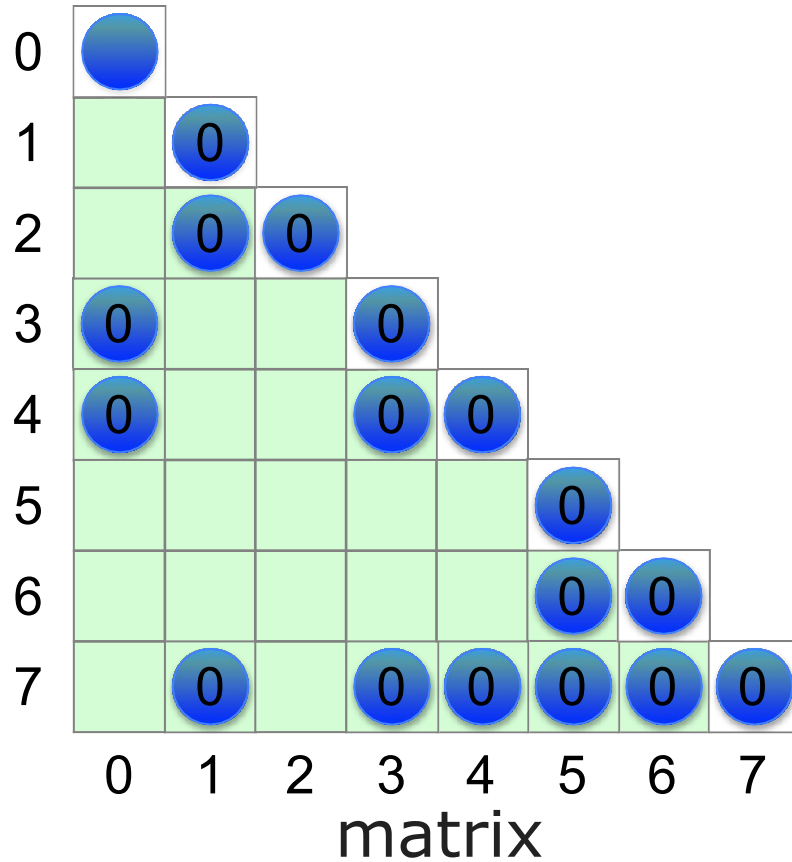


memory behavior

in-degree:

1	1	2	2	3	1	2	6
---	---	---	---	---	---	---	---

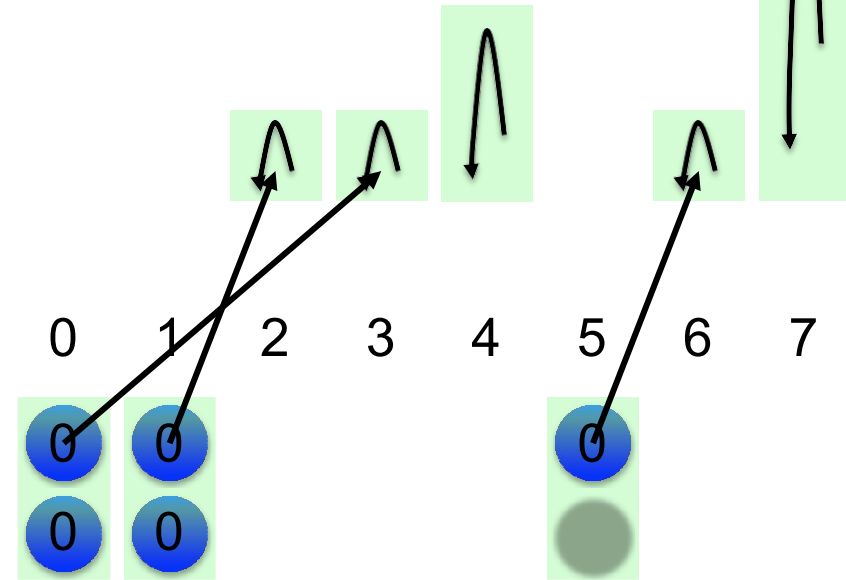
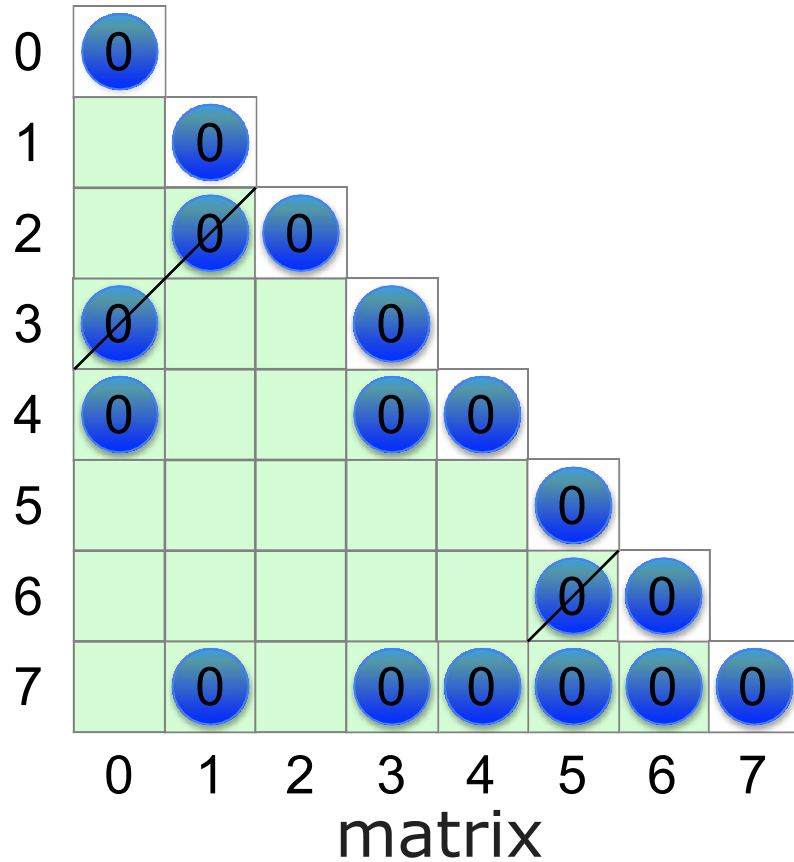
Cartoon of an example (step 3)



in-degree:

1	1	2	2	3	1	2	6
---	---	---	---	---	---	---	---

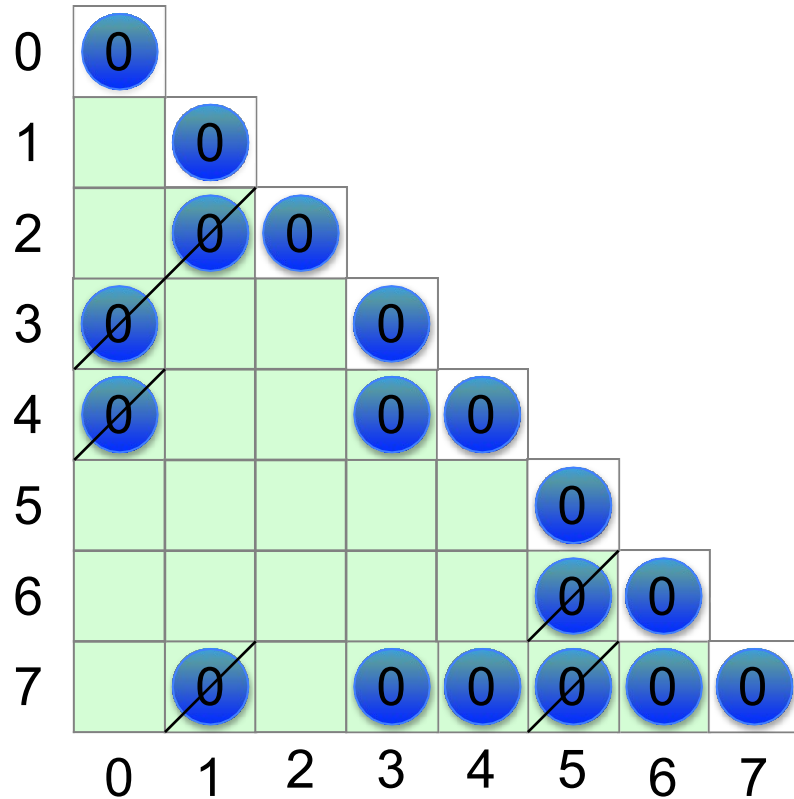
Cartoon of an example (step 4)



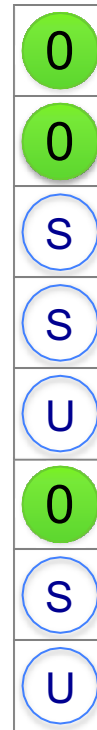
in-degree:

1	1	1	1	3	1	1	6
---	---	---	---	---	---	---	---

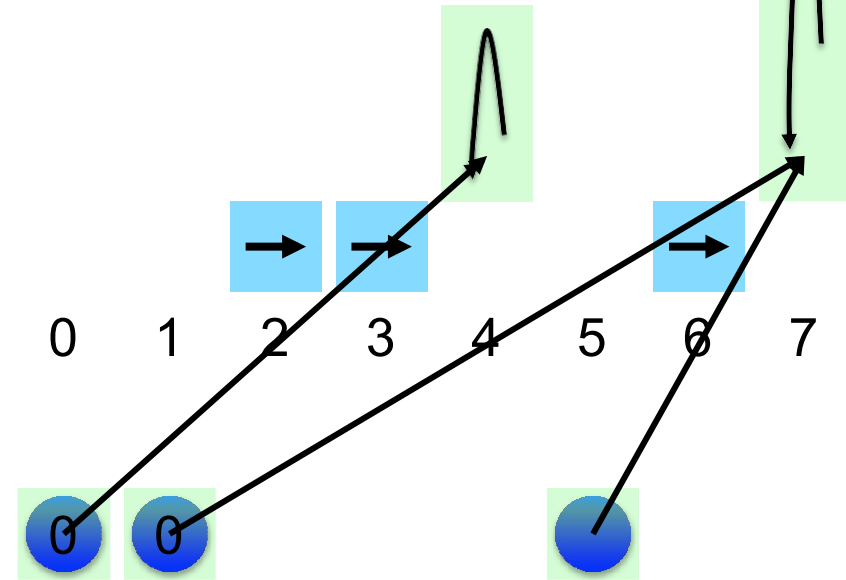
Cartoon of an example (step 5)



matrix



solution

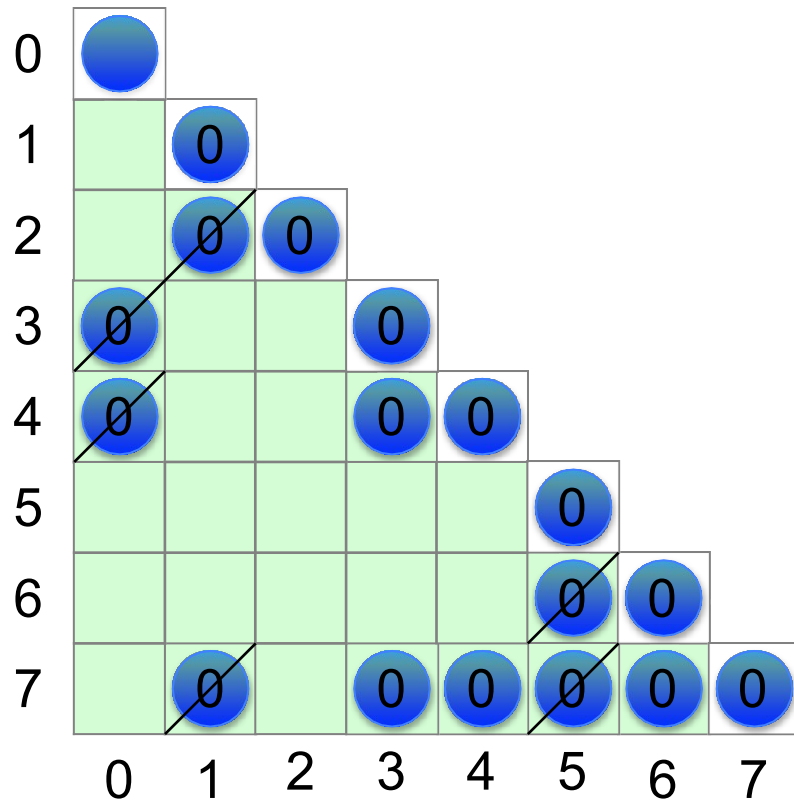


memory behavior

in-degree:

1	1	1	1	2	1	1	4
---	---	---	---	---	---	---	---

Cartoon of an example (step 6)



matrix



solution

0 1 2 3 4 5 6 7

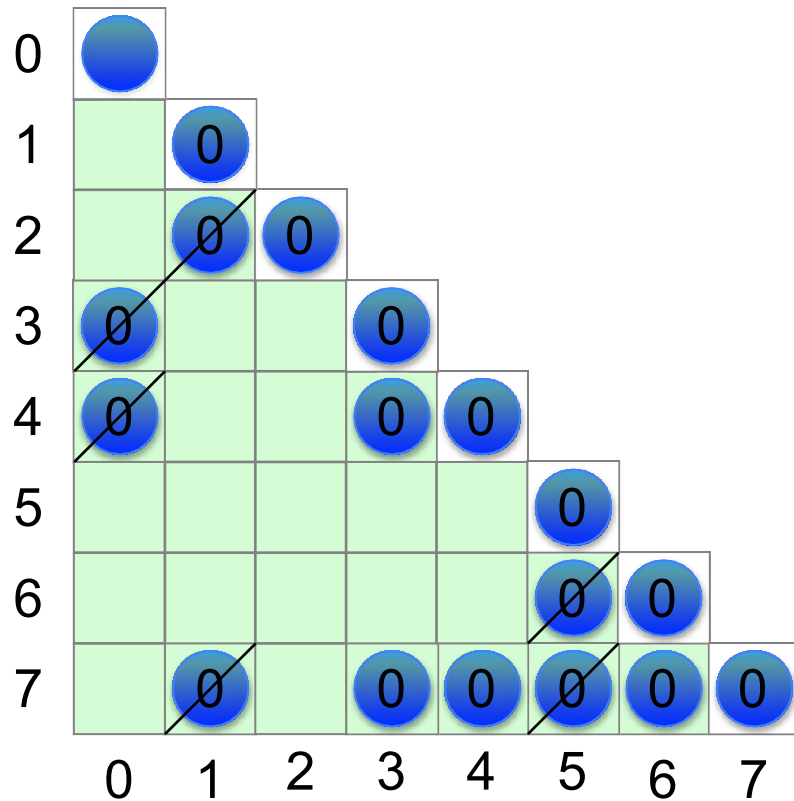


memory behavior

in-degree:

1	1	1	1	2	1	1	4
---	---	---	---	---	---	---	---

Cartoon of an example (step 7)



matrix



solution

0 1 2 3 4 5 6 7

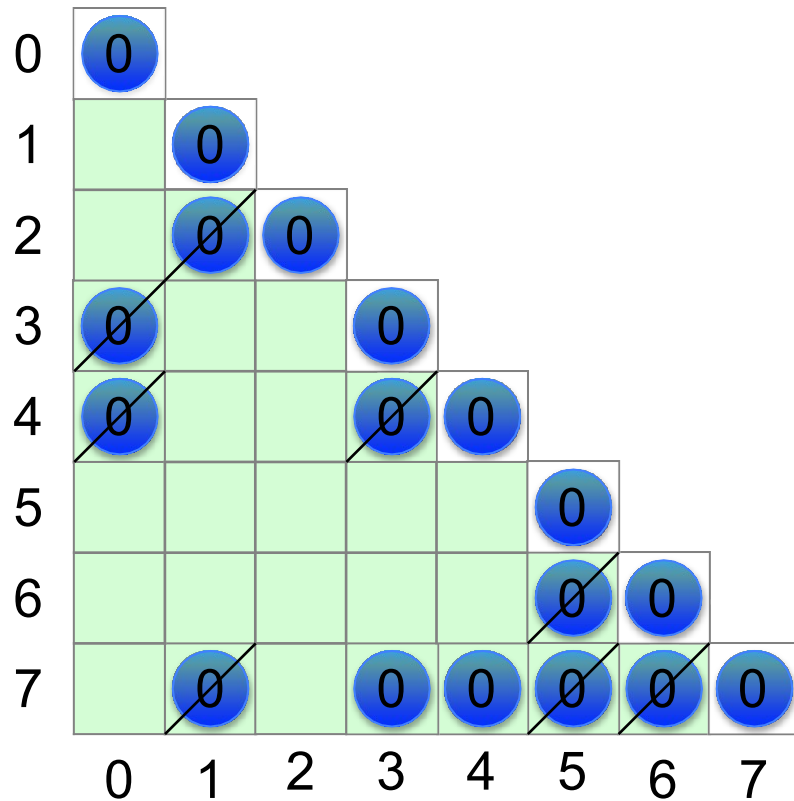


memory behavior

in-degree:

1	1	1	1	2	1	1	4
---	---	---	---	---	---	---	---

Cartoon of an example (step 8)

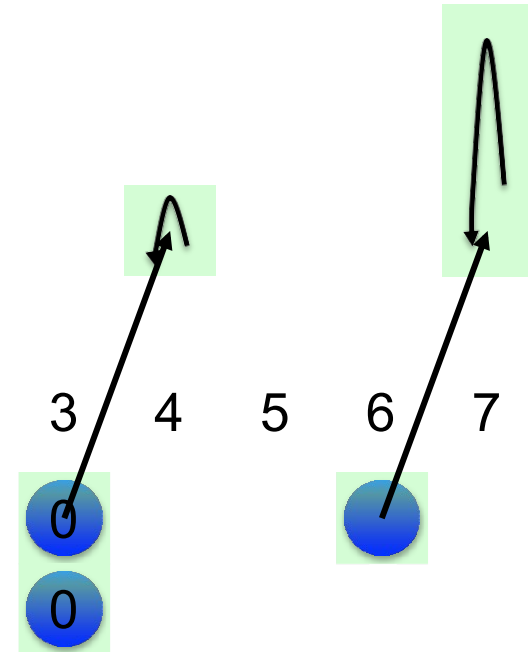


matrix



solution

0 1 2 3 4 5 6 7

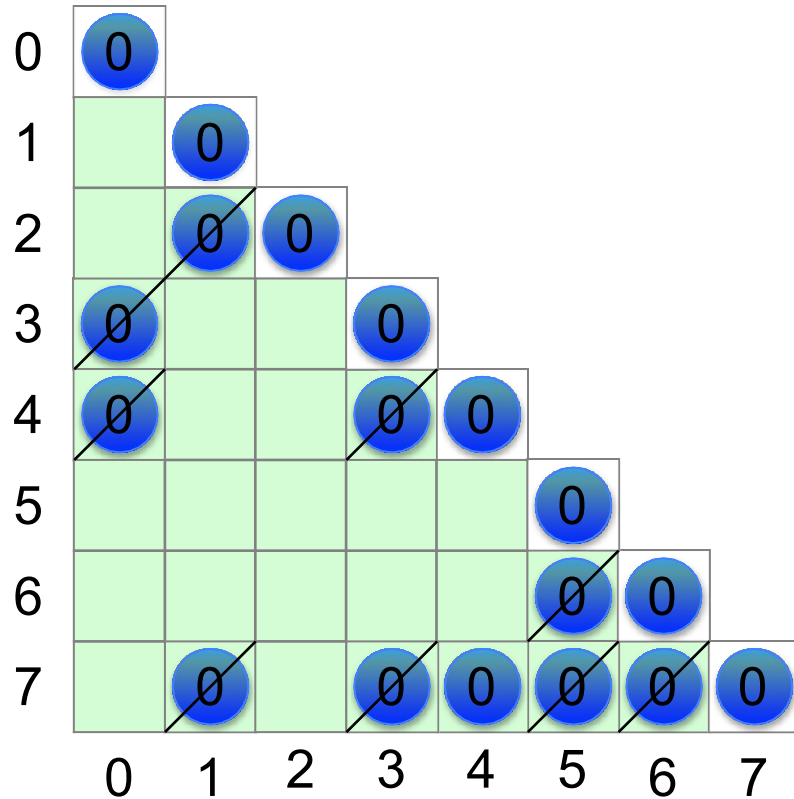


memory behavior

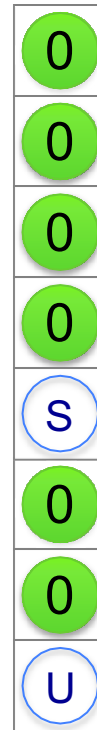
in-degree:

1	1	1	1	1	1	1	3
---	---	---	---	---	---	---	---

Cartoon of an example (step 9)

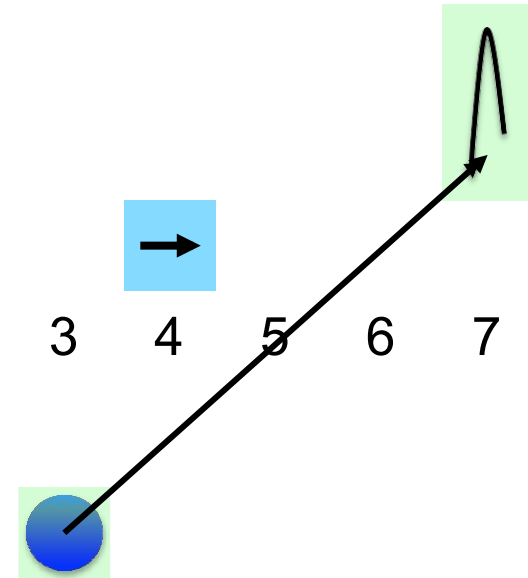


matrix



solution

0 1 2 3 4 5 6 7

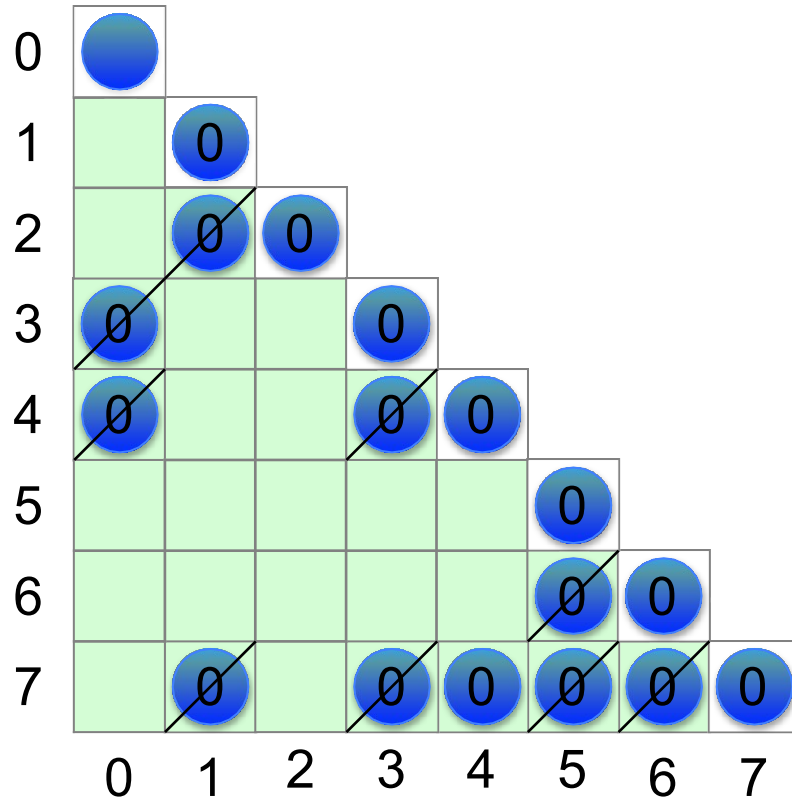


memory behavior

in-degree:

1	1	1	1	1	1	1	2
---	---	---	---	---	---	---	---

Cartoon of an example (step 10)



matrix



solution

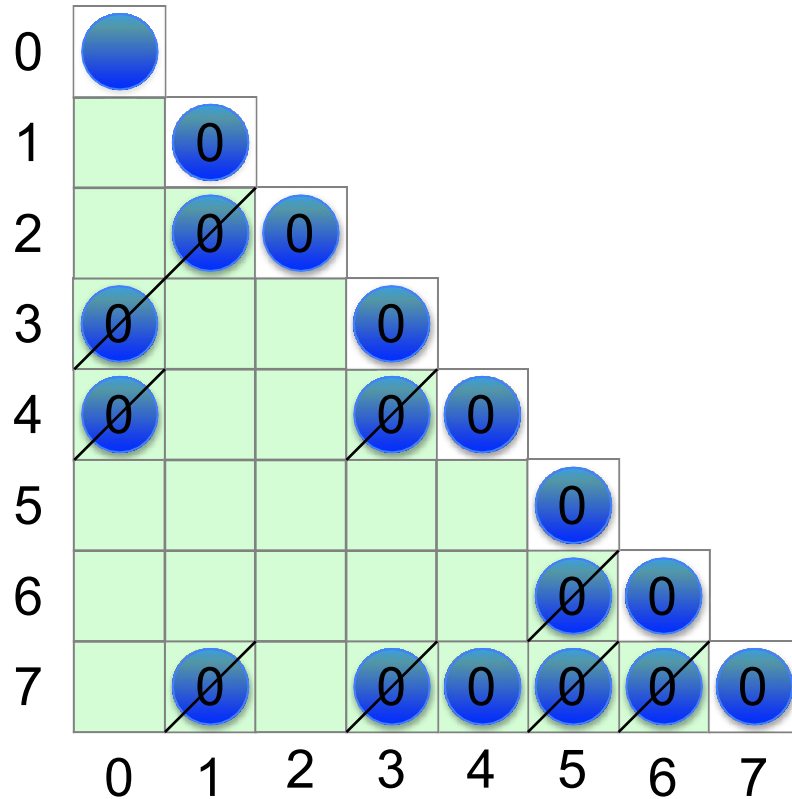
0 1 2 3 4 5 6 7

memory behavior

in-degree:

1	1	1	1	1	1	1	2
---	---	---	---	---	---	---	---

Cartoon of an example (step 11)



matrix



solution

0 1 2 3 4 5 6 7

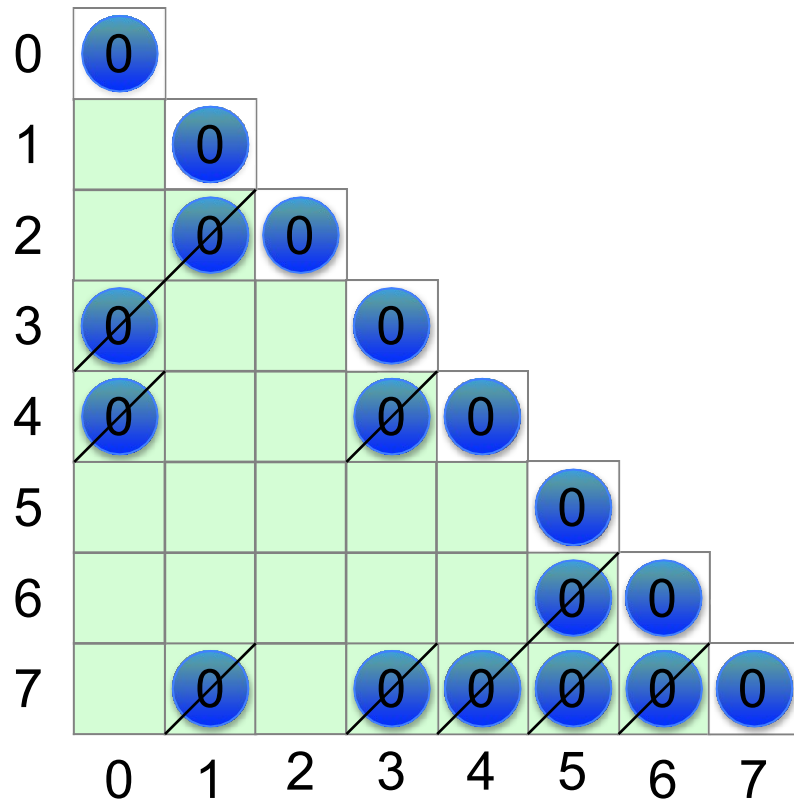


memory behavior

in-degree:

1	1	1	1	1	1	1	2
---	---	---	---	---	---	---	---

Cartoon of an example (step 12)

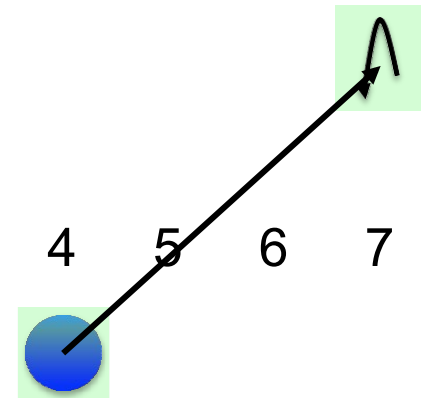


matrix



solution

0 1 2 3 4 5 6 7

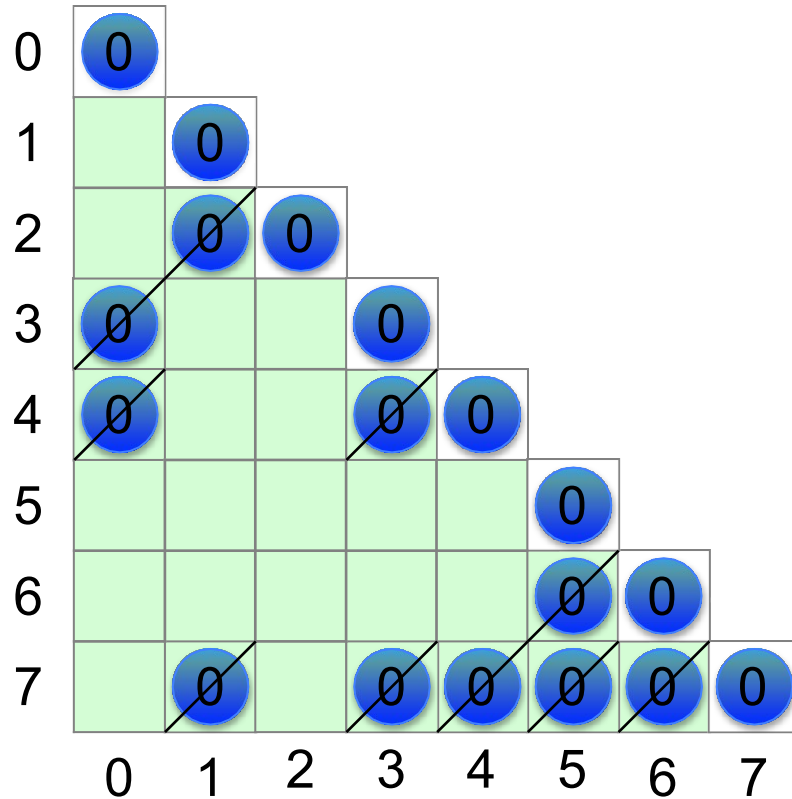


memory behavior

in-degree:

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

Cartoon of an example (step 13)



matrix



solution

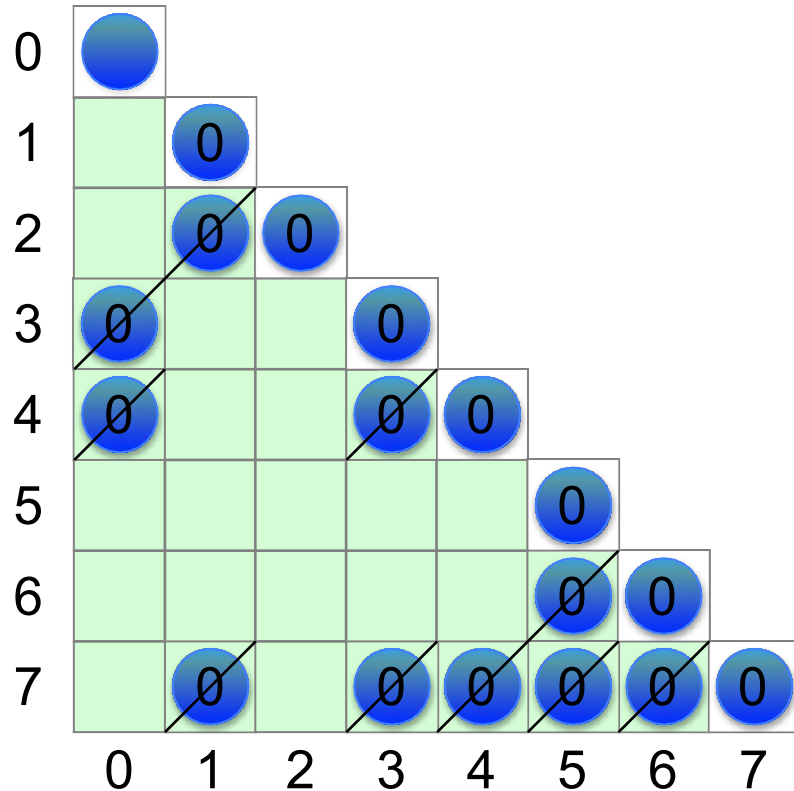
0 1 2 3 4 5 6 7

memory behavior

in-degree:

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

Cartoon of an example (step 14)



matrix



solution

0 1 2 3 4 5 6 7

memory behavior

in-degree:

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---



Advantages of the sync-free SpTRSV

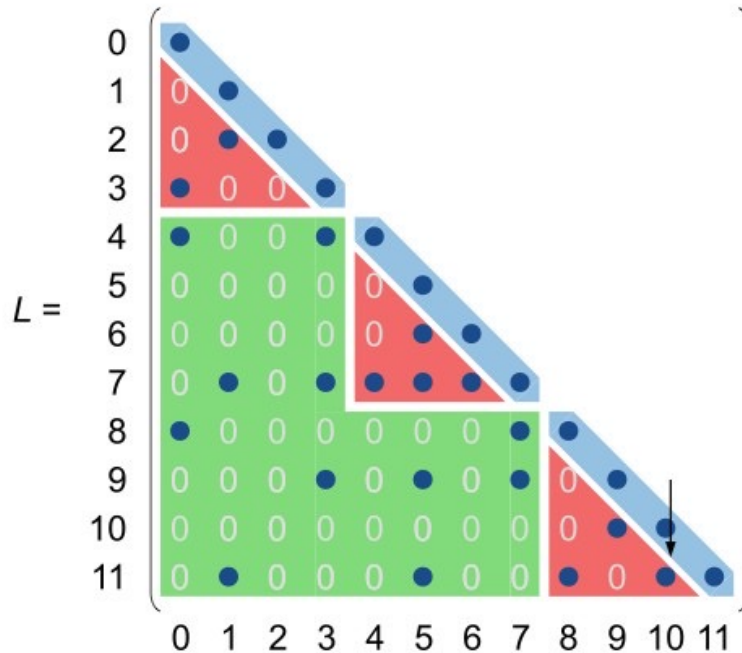
- 预处理阶段只需要生成in-degree信息，我们的例子中，in-degree信息如下：

1	1	2	2	3	1	2	6
---	---	---	---	---	---	---	---

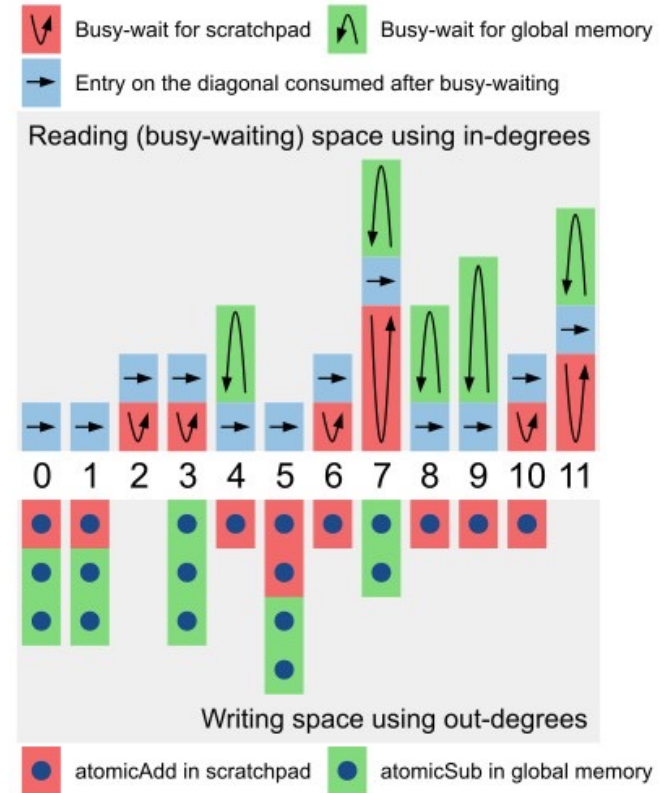
计算时间开销较低

- 在求解问题阶段，不需要显式的全局（device-level）
barrier 同步操作

See a more complex case in the paper



(a) Matrix.



(b) Read/write behaviours.



Experimental Results



Experiment platforms

- **Intel Xeon E5-2695 v3 (dual-socket)**, 2x 14 Haswell cores @ 2.3 GHz, 2x 68.3 GB/s Bandwidth. (1) *mkl_?csrtrsv* in MKL v11.3, (2) *inspector-executor mkl_sparse_?_trsv* in MKL v11.3.
- **nVidia Tesla K40c**, 2880 CUDA cores @ 0.75 GHz, 288 GB/s bandwidth. (1) *cusparsesolve?csrsv2_solve* in cuSPARSE v7.5, (2) **Sync-Free SpTRSV (this work)**.
- **nVidia GeForce GTX Titan X**, 3072 CUDA cores @ 1 GHz, 336.5 GB/s bandwidth. (1) *cusparsesolve?csrsv2_solve* in cuSPARSE v7.5, (2) **Sync-Free SpTRSV (this work)**.
- **AMD Radeon R9 Fury X**, 4096 Radeon cores @ 1.05 GHz, 512 GB/s bandwidth. (1) **Sync-Free SpTRSV (this work)**.



Benchmark suite

- 11 matrices downloadable from the University of Florida sparse matrix collection.

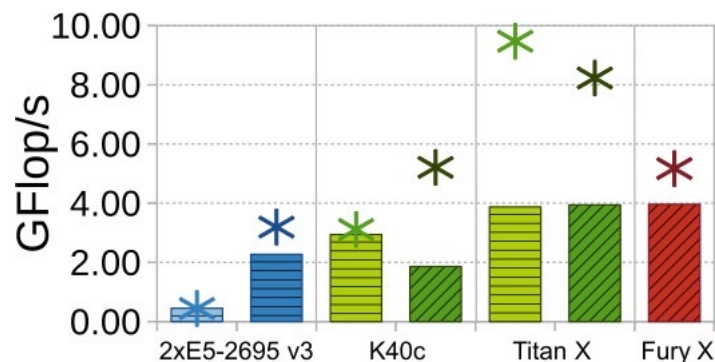
Matrix name	#Rows/Columns	#Nonzeros	#Level-sets	Parallelism
nlpkkt160	8,345,600	229,518,112	2	4,172,800
road_central	14,081,816	33,866,826	59	238,675
road_usa	23,947,347	57,708,624	77	311,004
webbase-1M	1,000,005	3,105,536	514	1,946
wiki-Talk	2,394,385	5,021,410	522	4,587
chipcool0	20,082	281,150	534	37
Dense	2,000	4,000,000	2,000	1
FEM/Cantilever	62,451	4,007,383	2,397	26
crankseg_1	52,804	10,614,210	4,056	13
FEM/ship_003	121,728	8,086,034	4,367	28
hollywood-2009	1,139,905	113,891,327	82,735	14

Performance (GFlop/s, FP32 & FP64)



Single precision SpTRSV

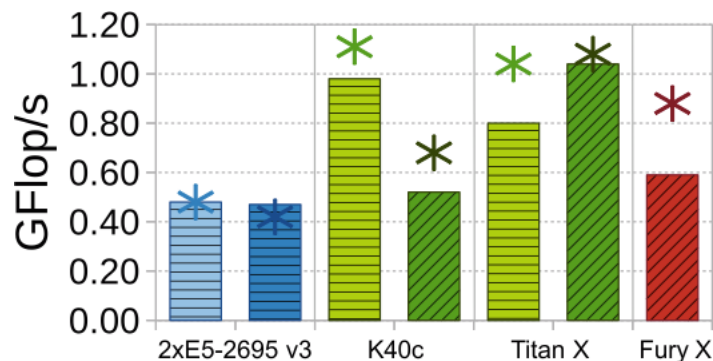
- * MKL v11.3 (serial routine)
- * MKL v11.3 (parallel routine)
- * Synchronization-Free method (AMD)
- * cuSPARSE v7.5
- * Synchronization-Free method (NVIDIA)



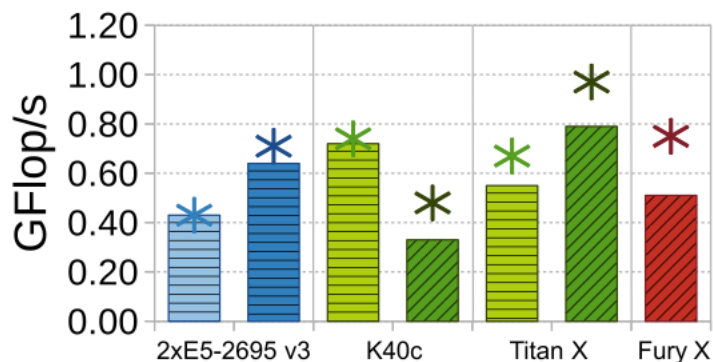
nlpkkt160

Double precision SpTRSV

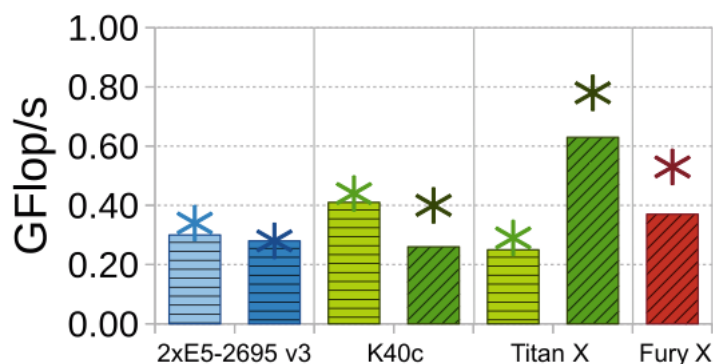
- * MKL v11.3 (serial routine)
- * MKL v11.3 (parallel routine)
- * Synchronization-Free method (AMD)
- * cuSPARSE v7.5
- * Synchronization-Free method (NVIDIA)



Dense



crankseg_1



hollywood-2009



Preprocessing cost (milliseconds)

Matrix name	Intel 2xE5-2695 v3	NVIDIA K40c		NVIDIA Titan X		AMD Fury X
	MKL	cuSPARSE	Sync-Free	cuSPARSE	Sync-Free	Sync-Free
nlpkt160	64.43	40.58	7.27	19.99	8.91	5.58
road_usa	155.48	160.41	5.06	84.01	3.37	2.31
road_central	92.16	82.01	9.28	42.62	6.98	5.53
wiki-Talk	17.38	16.27	0.33	10.49	0.20	0.16
webbase-1M	7.08	8.53	0.19	5.48	0.13	0.11
chipcool0	1.05	1.48	0.02	1.41	0.02	0.02
FEM/ship_003	9.14	6.41	0.19	4.34	0.26	0.13
FEM/Cantilever	9.52	8.92	0.10	8.28	0.16	0.07
hollywood-2009	223.54	139.98	5.20	204.10	4.82	2.78
crankseg_1	9.30	8.93	0.24	6.14	0.43	0.14
Dense	9.29	3.46	0.08	2.99	0.12	0.05
Harmonic mean	6.80	6.99	0.12	5.71	0.13	0.10



Conclusion

所提的synchronization-free SpTRSV:

- 移除了查找level-set的限制和运行时显式barrier同步的开销;
- on-chip (shared memory) 和off-chip memory 操作优化;
- 和level-set方法相比有较大性能提升, 特别是当#level-sets 数值较大时;
- 支持CUDA for NVIDIA GPUs 和 OpenCL for AMD GPUs.

谢谢！

